

Towards trustworthy agentic AI: a comprehensive survey of safety, robustness, privacy, and system security

Jinhu Qi¹, Muzhi Li¹, Jiahong Liu¹, Yuqin Shu¹, Dianzhi Yu¹, Shicheng Ma¹, Wenqian Cui¹, Yiyang Zhao^{2,3}, Yiyi Chen², Ruoxi Jiang^{2,3}, Irwin King^{1,*}, Zenglin Xu^{2,3,*}

Academic Editors: Vincent Hilaire, Soo Ling Lim

Abstract

Agentic AI systems—Large Language Models (LLMs) augmented with planning, tool use, memory, and long-horizon interactions—can execute complex tasks autonomously, but their multi-step trajectories introduce new failure modes that challenge trustworthiness. This survey provides a focused examination of trustworthy agentic AI through two core dimensions that are critical for high-risk deployments: Safety and Robustness and Privacy and System Security. For each dimension, we clarify key concepts, identify where risks emerge along the agent workflow, and summarize stage-targeted mitigation strategies. Other trustworthiness aspects (value alignment, transparency, fairness, and accountability) are discussed as relevant context rather than parallel chapters. To support consistent comparison and deployment decisions, we consolidate evaluation into a unified metrics-and-benchmarks hub, emphasizing both outcome and process signals (e.g., constraint violations, trace completeness, and adversarial success rates) and offering scenario-to-metric guidance for release gating. We conclude by outlining open challenges such as self-evolving agents, runtime monitoring and verification, privacy-preserving personalization, and the trust–utility trade-off, and present a case study of real-world security failures in open-source agentic systems (OpenClaw/Moltbook). Our goal is to serve as a practical reference for researchers and practitioners building trustworthy agentic systems in high-stakes environments.

Keywords: *agentic AI, trustworthiness, safety, robustness, privacy, system security, large language models, evaluation, benchmarks, multi-agent systems*

Citation: Qi J, Li M, Liu J, Shu Y, Yu D, Ma S, et al. Towards trustworthy agentic AI: a comprehensive survey of safety, robustness, privacy, and system security. *Academia AI and Applications* 2026;2. <https://doi.org/10.20935/AcadAI8260>

1. Introduction

1.1. Motivation

The paradigm shift from static Large Language Models (LLMs) to agentic systems—capable of autonomous planning, tool invocation, and multi-step reasoning—has enabled their deployment in critical real-world applications. From automating complex software development cycles to serving as intelligent intermediaries in healthcare and financial services, these agents leverage their ability to interact with external environments to fulfill high-level goals [1–3]. This increasing autonomy transforms them from mere productivity aids into central nodes of modern digital infrastructure.

However, as LLM-based assistants are increasingly connected to enterprise data and tools, failures can translate into direct real-world impact. For example, a “zero-click” prompt injection vulnerability in Microsoft 365 Copilot (CVE-2025-32711, “EchoLeak”) was publicly reported and patched, highlighting that specially crafted untrusted inputs (e.g., emails) may trigger unintended behaviors and enable sensitive data exposure without explicit user interaction [4, 5]. More broadly, prior work has shown that indirect prompt injection blurs the boundary between data and instructions in LLM-integrated applications, allowing attacker-controlled content retrieved from the web or documents to hijack

tool-using systems and cause data exfiltration or unintended actions [6–8]. These incidents underscore that “trustworthiness” for agentic AI must be assessed at the system level, beyond single-turn outputs.

Against this backdrop, Large Language Models (LLMs) have rapidly progressed from pure text generators to systems that can act in the world. Modern “agentic AI” augments LLMs with planning, tool use (e.g., web browsing, APIs, and code execution), memory, and long-horizon interactions, enabling them to decompose complex goals into executable steps and iteratively refine behavior from feedback [9]. Representative systems demonstrate that such agents can autonomously explore environments and continuously acquire skills (e.g., via lifelong interaction and self-improvement loops) [10–12]. This shift from static single-turn models to autonomous or semi-autonomous agents is a capability leap—but it also introduces qualitatively new risks.

Unlike traditional predictive models or chat-based LLMs, agentic systems produce multi-step trajectories whose intermediate states (plans, tool calls, retrieved evidence, and memory updates) can

¹Department of Computer Science and Engineering, Faculty of Engineering, The Chinese University of Hong Kong, Hong Kong, China.

²Artificial Intelligence Innovation and Incubation Institute, Fudan University, Shanghai, China.

³Shanghai Academy of AI for Science, Shanghai, China.

*email: king@cse.cuhk.edu.hk (I.K.); zenglinxu@fudan.edu.cn (Z.X.)

directly affect real-world outcomes. A minor error early in a trajectory can cascade into high-impact actions, and the agent's interaction with tools expands the attack surface (prompt injection, tool misuse, and data exfiltration) beyond what is captured by conventional LLM safety evaluations. Moreover, agents increasingly operate in settings where human oversight is intermittent rather than continuous; this raises fundamental questions about the accountability, auditability, and interruptibility of agent behavior [13, 14]. As agent autonomy grows, “trustworthiness” must be assessed not only by final outputs, but also by process signals such as constraint compliance, trace evidence, and robustness to adversarial and long-horizon stress.

At the same time, trustworthiness itself is not a single property. It spans multiple dimensions that interact with one another: strengthening memory can improve effectiveness but may increase privacy risk; adding safeguards can reduce catastrophic failures but may reduce utility or increase cost; and explanation interfaces can improve auditability but may also inflate over-trust if explanations are unfaithful. These tensions motivate a survey that treats trustworthy agentic AI as a system-level problem rather than a model-only problem and that makes evaluation comparable across dimensions and deployment scenarios.

1.2. Scope and perspective

We focus on LLM-based agentic systems that (i) plan over extended horizons, (ii) use external tools and environments, (iii) may incorporate memory, self-reflection, or multi-agent interaction. To structure the discussion, we adopt an agent workflow lens—Perceive → Plan → Act → Reflect → Learn—to pinpoint where risks arise and where mitigations intervene. This workflow is not meant to be a strict architecture requirement; rather, it provides a consistent interface for mapping threats, defenses, and evaluation signals across diverse agent designs.

1.3. Relationship to chat-based system risks

Many trust and safety concerns discussed in this survey—such as harmful content generation, deceptive outputs, or inappropriate advice—also arise in non-agentic, chat-based LLM systems. We do not exclude these foundational risks; rather, we emphasize that agentic autonomy amplifies and extends them in qualitatively new ways. For example, prompt injection attacks against a chat-based system may produce misleading text, but the same attack against a tool-using agent can trigger unauthorized code execution, data exfiltration, or irreversible real-world actions. Similarly, harmful persuasion in a chat setting is bounded by the conversation, whereas an agent with tool access can autonomously act on such persuasion across a multi-step trajectory. The risks and mitigations we survey are therefore not exclusive to agentic systems, but their severity, attack surface, and cascading potential are substantially greater when agents operate with real-world affordances [15]. Where applicable, we note when a risk or method also applies to non-agentic LLM deployments.

1.4. Illustrative rather than exhaustive scope

Given the rapidly evolving nature of agentic AI, the risks and mitigation methods discussed in this survey should be understood as illustrative examples reflecting the current state of knowledge, rather than a provably exhaustive enumeration. New risks and

solution methods are likely to emerge as agent architectures, tool ecosystems, and deployment contexts continue to evolve. Where possible, we note the boundaries of current understanding and highlight directions where coverage remains incomplete.

1.5. Comparison with existing surveys

Prior surveys have investigated trustworthy AI principles and requirements for general AI systems (e.g., [16]) and trustworthiness evaluation for Large Language Models (e.g., [17, 18]). Recent works start to address trust/safety issues for LLM-based agents and multi-agent systems (e.g., [19, 20]) or provide architecture/application-centered overviews of agentic AI (e.g., [21, 22]). In contrast, this survey integrates a multi-dimensional trust taxonomy with a workflow lens, and further consolidates process-aware evaluation and scenario-based release gating (**Table 1**).

Table 1 • Comparison with representative related surveys on trustworthy AI/large language models/agents.

Survey	Scope	MDT	WL	EH	RG
[16]	General TAI	✓	—	~	—
[17]	Trustworthy LLMs	✓	—	~	—
[18]	LLM benchmark	✓	—	✓	—
[19]	LLM agents/MAS	~	~	~	—
[20]	LM & agents (safety)	~	—	✓	—
[21]	Agentic AI (arch.)	~	~	—	—
[22]	Tool-learning agents	~	~	~	—
This survey (ours)	Trustworthy agentic AI	✓	✓	✓	✓

Column abbreviations: MDT: Multi-dimensional trust taxonomy; WL: Workflow lens (perceive–plan–act–reflect–learn); EH: Evaluation hub (consolidated metrics and benchmarks); RG: Release gating (scenario-to-metric guidance). Symbols: ✓: explicit main focus, ~: partial coverage, —: not a primary focus.

1.6. Contributions

Our main contributions are as follows:

- A focused examination of two core trustworthiness dimensions. We concentrate on Safety and Robustness and Privacy and System Security—two dimensions that are particularly critical for high-risk agentic AI deployments. Other trustworthiness aspects (value alignment, transparency, fairness, and accountability) are discussed as relevant context rather than parallel chapters.
- Risk-to-method mapping along the agent workflow. For each core dimension, we provide a consistent Definition → Risks → Methods structure, illustrating where failures can occur in the agent lifecycle and summarizing representative stage-targeted mitigations (e.g., constrained optimization, red teaming, runtime shielding, and sandboxing). These mappings are intended to be illustrative of the current landscape rather than exhaustive.
- A consolidated evaluation hub for metrics and benchmarks. Because evaluation content is often fragmented across sub-fields, we consolidate cross-dimension metrics and representative benchmark families into Section 4. We emphasize both

outcome metrics (e.g., success and catastrophic risk) and process metrics (e.g., constraint violation rates and trace coverage), and provide scenario-to-metric guidance for practical release gating in high-risk scenarios.

- Open challenges and solutions. We identify research frontiers such as self-evolving agents under concept drift, verification and runtime monitoring for rare events and interactive settings, privacy-preserving personalization, and the trust–utility trade-off.

1.7. Paper organization

Figure 1 illustrates the overall structure and provides a reading guide. The survey is organized into four main parts:

1. Preliminaries (Section 2): defines agentic AI and its components along the five-stage workflow (Perceive–Plan–Act–Reflect–Learn; see **Figure 2** for the system architecture), reviews reinforcement learning and preference optimization foundations relevant to trustworthy training, and outlines
2. Core dimensions (Section 3): examines the two core trustworthiness dimensions—Safety and Robustness and Privacy and System Security—each following a consistent Definition → Risks → Methods structure aligned to the agent lifecycle (see **Figure 3** for a hierarchical taxonomy with stage-specific mitigation methods and key references).
3. Evaluation Hub (Section 4): consolidates metrics, benchmark suites, scenario-to-metric mapping for release gating, and recommended evaluation pipelines and reporting standards into a unified reference.
4. Challenges and conclusion (Section 6): discusses open challenges (self-evolving agents, monitoring, personalization, and the trust–utility trade-off) and presents a case study on security failures in open-source agentic systems (Open-Claw/Moltbook).

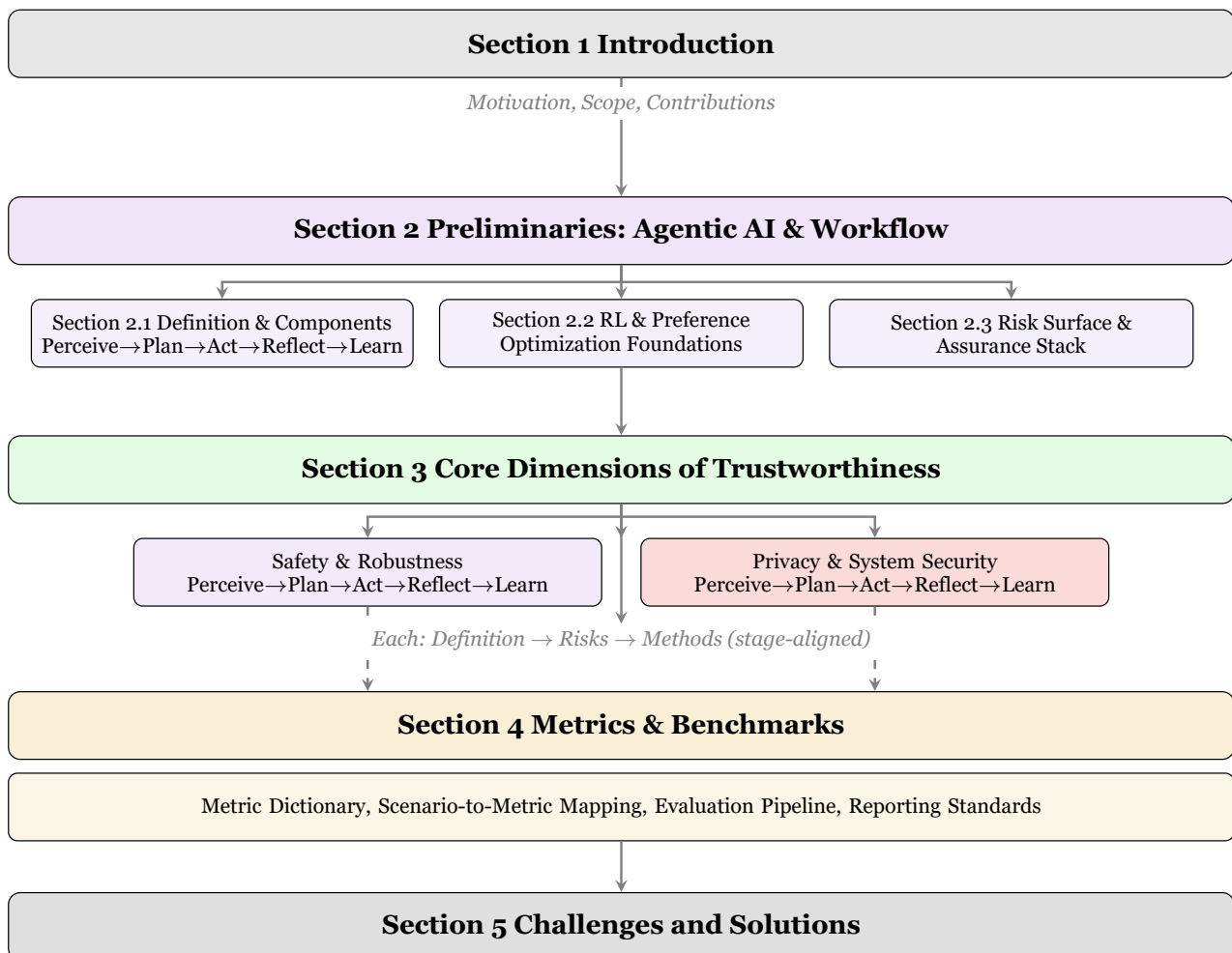


Figure 1 • Paper structure and reading guide. The survey begins with motivation and agentic AI preliminaries (Sections 1 and 2), then presents two core trustworthiness dimensions—Safety and Robustness and Privacy and System Security—with consistent Definition→Risks→Methods structure (Section 3). Evaluation metrics consolidate process- and outcome-level assessments (Section 4). Challenges and solutions conclude the survey (Section 6). Gray boxes represent introductory and concluding sections; purple boxes denote preliminaries; green boxes denote core dimensions; yellow boxes denote the evaluation hub. Solid arrows indicate the primary reading flow; dashed arrows indicate dependency relationships between dimensions and evaluation.

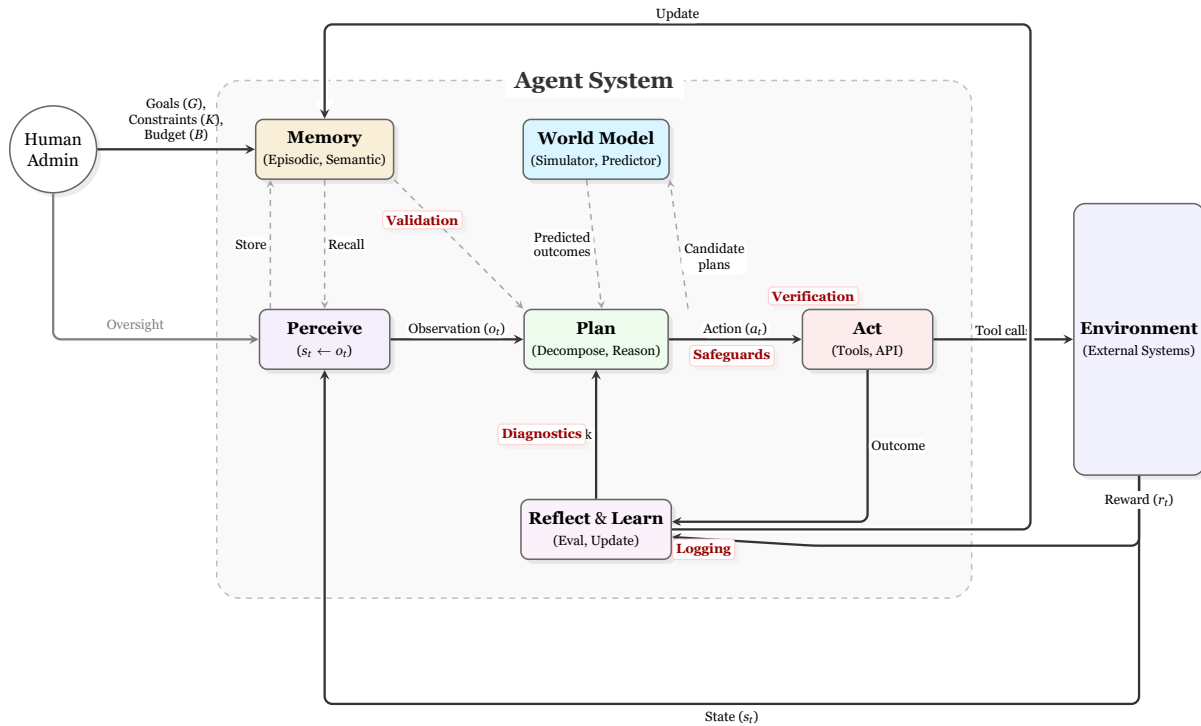


Figure 2 • Agentic AI system architecture. The agent receives goals (G), constraints (K), and budgets (B) from human oversight. The Environment emits the current state (s_t) to Perceive, which produces an observation (o_t) passed to Plan. Plan selects an action (a_t) for Act, which invokes tools and receives outcomes and rewards (r_t) that feed into Reflect and Learn (Stages 4 and 5). The World Model supports planning by simulating candidate plans and returning predicted outcomes. Memory stores and recalls episodic/semantic context across stages. Dashed lines denote data flow; solid lines denote the primary control flow. Red labels denote assurance hooks—control points for risk mitigation (defined in **Table 2**).

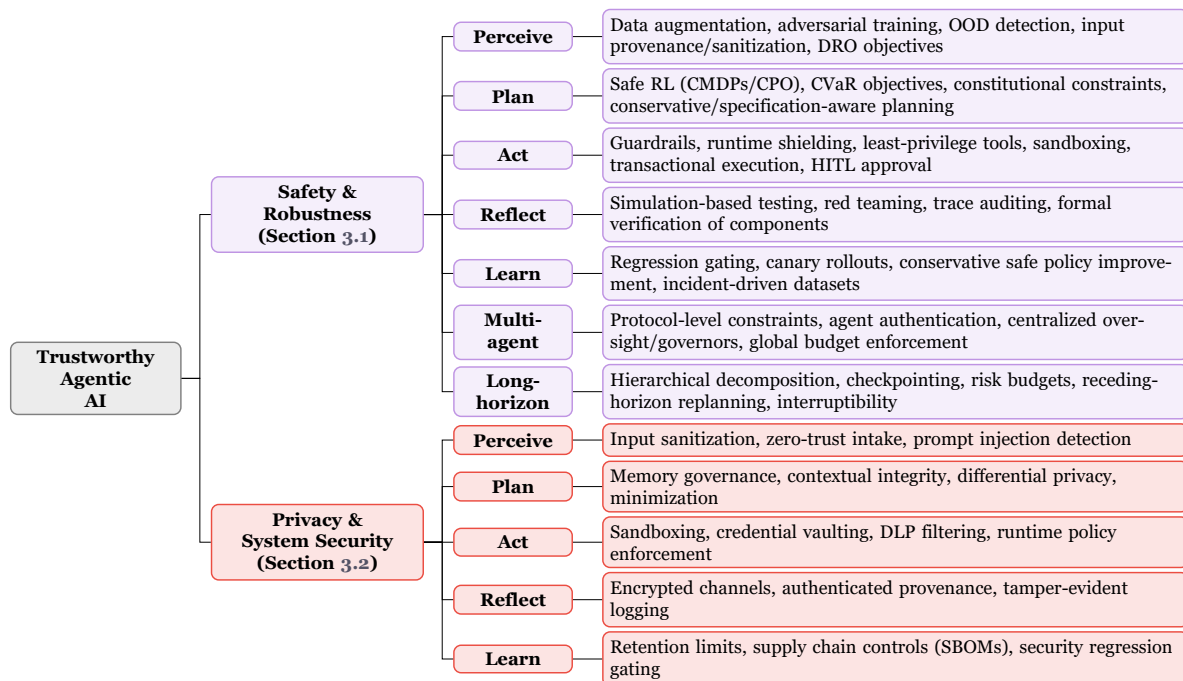


Figure 3 • A hierarchical taxonomy of trustworthy agentic AI organized along two core trustworthiness dimensions: Safety and Robustness and Privacy and System Security. Each dimension is decomposed according to the five-stage agent lifecycle (Perceive→Plan→Act→Reflect→Learn). The Safety and Robustness dimension additionally includes Multi-agent and Long-horizon branches, which address emergent coordination risks and compounding error beyond the single-agent, single-episode setting (see Sections 3.1.2 and 3.1.3). Purple nodes denote Safety and Robustness mitigations; red nodes denote Privacy and System Security mitigations; the gray root node represents the overarching trustworthy agentic AI concept. Leaf nodes list representative mitigation methods, ordered consistently with the text in Section 3. Abbreviations: OOD, out-of-distribution; DRO, distributionally robust optimization; CMDPs, constrained Markov decision processes; CPO, constrained policy optimization; CVaR, conditional value-at-risk; HITL, human-in-the-loop; DLP, data loss prevention; SBOMs, software bills of materials. Key references for each leaf are cited here: [23–50].

Table 2 • Assurance hooks across the agent workflow (corresponding to red labels in **Figure 2**).

Hook	Stage	Mechanism/control type
Validation	Perceive	Input filters (jailbreak detection), RAG citation checks.
Verification	Plan	Logic/formal verification of plans against constraints (K).
Safeguards	Act	Runtime shields, permission gates, and tool sandboxing.
Diagnostics	Reflect	Anomaly detection (OOD/drift), value-alignment checks.
Logging	All	Tamper-evident traces for audit and accountability.

Abbreviations: RAG: retrieval-augmented generation; OOD: out-of-distribution.

2. Preliminaries

2.1. Agentic AI: definition, components, and workflow

2.1.1. Definition

We define agentic AI as an AI system with persistent objectives that can perceive its environment, plan over multiple steps, and act via tools or actuators to affect external systems, while reflecting on outcomes and adapting its internal state under explicit human oversight, privacy/security policies, and operational constraints. This view follows the classical agent perspective in AI and RL [51–53] and is instantiated in recent LLM-based agents that interleave reasoning and acting [54, 55]. Unlike single-turn LLM responders, agentic systems close the loop between observation and consequences, which foregrounds Safety and Robustness and Privacy and System Security (with accountability as a supporting concern), and motivates process-aware evaluation and auditability.

2.1.2. Core components

A minimal, implementation-agnostic stack includes the following (each item notes trust relevance):

- Task and constraint specification (Goals G , Constraints K): formal goals, hard/soft constraints (safety, privacy, security), and budgets B ; anchors governance and safety/security-by-design [23, 51].
- Perception: state estimation s_t from observations o_t and knowledge access (Retrieval-Augmented Generation (RAG), function calls) with input validation [52, 56].
- World model/simulator: predictive models for consequence forecasting and counterfactual testing [57].
- Policy and planning (hierarchical): goal decomposition and multi-level control linking abstract plans to executable actions [54, 58].
- Value/reward model: task returns, preferences, and normative signals (details deferred to Section 2.2).
- Memory: short-term context and long-term episodic/semantic stores for continuity and credit assignment, with retention, access control, and minimization policies for sensitive data [10, 55].
- Tools and actuators: API/system/robotic calls mediated by least-privilege permissions, credential/secret scoping, and execution sandboxes [59].

- Runtime safeguards: anomaly monitors, safety constraint checkers, security policy enforcement (e.g., tool permissions, Data Loss Prevention (DLP)/redaction hooks), and rollback hooks for pre-/post-execution assurance [23].
- Human-in-the-loop interface: alert, approval, and takeover channels that define roles and escalation policies [23].
- Telemetry and audit logs: versioned traces for reproducibility, post hoc analysis, and accountability, with privacy-aware logging (redaction/Personally Identifiable Information (PII) minimization) when required [23].

2.1.3. Perceive → Plan → Act → Reflect → Learn

The agentic loop comprises five recurring stages (see **Figure 2**, **Tables 2** and **3**): (1) Perceive—ingest observations and external knowledge, quantify uncertainty and detect Out-of-Distribution (OOD) inputs [52, 56]. (2) Plan—generate and evaluate multi-step plans subject to constraints/budgets, optionally via model-based rollouts or receding-horizon control [57, 60]. (3) Act—execute tool/API/actuator calls; pre/post checks enforce constraints, tool permissions, and data-handling policies, and capture traces [59]. (4) Reflect—self-evaluate outcomes, detect errors and goal/value conflicts, and schedule human confirmation when confidence drops [55, 61]. (5) Learn—update policy, value, memory, and retrieval indexes offline/online with safe update rules and retention/expiration policies for sensitive data (Section 2.2) [10, 53]. This loop exposes assurance hooks at each boundary (validation, verification, safeguards, diagnostics, and logging) that integrate with the unified evaluation framework in Section 4.

Table 3 • Terminology and symbols used in Section 2.

Symbol	Term	Meaning/example
s_t	state	latent environment condition at time t
o_t	observation	sensor/IO evidence used to infer s_t
a_t	action	tool/API/actuator invocation
r_t	reward/value	task return or preference signal
τ	trajectory	sequence $(o_{0:T}, a_{0:T-1}, r_{0:T-1})$
G	goal/instruction	task specification or objective description
K	constraints	hard/soft limits (safety, policy, ethics)
B	budget	limits on tokens/latency/money/energy

Abbreviations: IO: input/output; API: application programming interface.

2.1.4. How agents differ from single-turn LLMs

Single-turn LLMs neither maintain persistent goals nor directly act on external systems. Agentic AI adds (i) long-horizon decision making with side effects, (ii) tool-mediated interventions with permissions and budgets, (iii) explicit uncertainty handling and recovery, (iv) operational evidence for audit, (v) expanded privacy/security exposure via memory and tool credentials that must be protected (Section 3.2) [54, 59, 62]. These differences motivate

standardized components and workflows in this preliminaries section.

2.2. Reinforcement learning foundations and preference optimization for agentic AI

2.2.1. Formalization: MDP and POMDP

We model single-agent agentic decision making as a Markov Decision Process (MDP), defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' | s, a)$ is the state transition function, $r(s, a)$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor controlling the trade-off between immediate and future rewards. The agent's objective is to find a policy π that maximizes the expected discounted return $J(\pi) = \mathbb{E}_{\pi} [\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)]$. When the full state is not directly accessible (as is common in real-world agent deployments), we extend to a Partially Observable MDP (POMDP) $(\mathcal{S}, \mathcal{A}, P, r, \gamma, \mathcal{O}, Z)$, where $\mathcal{O}$ is the observation space and $Z(o | s, a)$ is the emission (observation) function specifying the probability of receiving observation o given state s after action a . The agent then maintains a belief state and plans under uncertainty [52, 53, 63].

This formalization primarily applies to agents trained or fine-tuned using reinforcement learning (RL). While some agentic systems rely on prompt-based or in-context reasoning alone, RL-based agents are the focus of this subsection because RL provides the mathematical substrate for long-horizon optimization under uncertainty and constraint satisfaction—both of which are central to trustworthiness.

Multi-agent settings. When multiple agents interact, the single-agent MDP formalization no longer suffices; the appropriate model is a Markov Game (also called a stochastic game), which extends MDPs with joint action spaces and agent-specific reward functions [64]. We discuss multi-agent trustworthiness risks where relevant (e.g., Sections 3.1 and 3.2) but note that formal treatment of multi-agent RL is beyond the scope of this survey.

2.2.2. Training families and their trust implications

Different RL paradigms offer distinct advantages and risks for trustworthiness:

- Value-based and actor-critic methods: Standard on-policy (PPO [65]) and off-policy (SAC, DQN [66]) algorithms provide asymptotic optimality but are prone to reward hacking—exploiting misspecified objectives in unsafe ways. From a trust perspective, they require rigorous reward shaping and constraint satisfaction to ensure safety during the exploration phase.
- Offline RL: By learning exclusively from static datasets (logged trajectories) [67, 68], offline RL eliminates the physical risks of online exploration, making it ideal for safety-critical domains like healthcare or industrial control. However, it introduces distributional shift risks: agents may behave unpredictably if the deployment environment diverges even slightly from the training data coverage.
- Hierarchical RL (HRL): Decomposing tasks into high-level goals (options) and low-level controls [58, 69] directly supports interpretability and human oversight. Human operators can

audit or intervene at the high-level goal setting (e.g., “navigate to room A”) without needing to parse low-level motor commands, aligning well with the Goal (G) and Planning components of the agentic architecture.

- Model-based RL (MBRL) and MPC: Learning an explicit world model (dynamics model) [70, 71] enables lookahead planning and counterfactual reasoning, allowing agents to simulate the consequences of actions before execution. This provides a natural mechanism for runtime safety checks (e.g., rejecting unsafe trajectories), though trust is then contingent on the calibration and robustness of the learned model against OOD states.

2.2.3. Safe RL as constrained decision making

Safety can be formalized via constrained MDPs (CMDPs) [24], optimizing $\max_{\pi} J(\pi)$ s.t. $J_{c_i}(\pi) \leq d_i$ with Lagrangian or primal-dual updates (e.g., Constrained Policy Optimization (CPO)) [25] and complemented by runtime monitors and verification/shielding [26, 72]. Practically, training-time guarantees reduce the frequency of violations, while runtime safeguards catch residual risks.

2.2.4. Preference/alignment optimization

To align objectives with human or normative signals, modern agents use preference-based training: Reinforcement Learning from Human Feedback (RLHF) combines learned rewards from comparisons with policy optimization [73–76]. RLAIIF replaces human labels with AI feedback and constitutional rules [32]. Recent work has also explored value alignment through multi-objective RL [77], embedding ethical constraints into RL environments [78], and encoding norms for value-aligned behavior [79]. Beyond RL-based pipelines, direct preference optimization (DPO and related PO methods) optimizes policies against preference data without an explicit reward model [80, 81]. These methods trade off stability, sample efficiency, and generalization; open issues include preference drift, norm conflict, and evaluation fidelity [82]. In this survey, we unify RLHF/RLAIIF and direct PO variants as a preference optimization family; varying implementations (e.g., DPO- or KTO-style objectives) can be plugged into the Perceive–Plan–Act–Reflect–Learn loop (Section 2.1) and evaluated using Section 4.

2.2.5. Takeaway

RL supplies the mathematical substrate for long-horizon, uncertain, tool-mediated behavior; preference optimization supplies the normative signal that turns competence into trustworthy competence. In high-risk deployments, this competence must be coupled with risk controls: constraint/risk-aware learning for Safety and Robustness (Section 3.1) and privacy/security-aware data handling when learning from logs, memories, or tool traces (Section 3.2).

2.3. Trustworthiness risk surface and assurance stack for agentic AI

2.3.1. Risk surface and threat model

We formalize the risk surface via specific threat actors and failure modes that motivate our core trustworthiness dimensions.

- **Security Threat Model (Attackers and Goals):** Security risks arise from adversarial actors targeting Confidentiality, Integrity, or Availability (CIA).
 - **Attacker Profiles:** (i) Malicious Users (jailbreaking, prompt injection); (ii) External Web/Environment (indirect injection via retrieval); (iii) Supply Chain (compromised tools/models); (iv) Insiders (unauthorized admin access).
 - **Capabilities:** Ranging from black box (prompt access only) to white box (deployment infrastructure access).
- **Safety Failure Modes (Unintended Harm):** Safety risks arise from system malfunctions or generalization failures, even without malice.
 - **Specification Error:** Reward hacking, unsafe side effects, or proxy-gaming (e.g., Over-Opt).
 - **Distributional Shift:** Perception/planning failures on OOD inputs.
 - **Compounding Error:** Small variations in early steps accumulating into unsafe trajectories (butterfly effect).
 - **Tool Control Failure:** Inability to halt or revoke unsafe tool execution (e.g., mass file deletion).
 - **Catastrophic Event Definition:** Any irreversible harm (physical, financial, reputational) exceeding a defined severity threshold (aligned with CER in Section 4).

2.3.2. Assurance stack (before, during, after training; and at runtime)

Trustworthy deployment requires layered assurance mechanisms that span the full lifecycle of an agentic system. No single layer is sufficient on its own: each addresses a different class of failure and compensates for gaps in the others, forming a defense-in-depth stack with four complementary tiers.

Upfront (pre-deployment) assurance establishes the safety envelope within which the agent is expected to operate. This includes formalized requirements and hazard analysis, data and environment design, red teaming, static checks, and simulation certification (including domain randomization) [27, 83]. These efforts produce the threat models, operational constraints, and test suites that anchor all subsequent tiers.

Building on this foundation, training-time assurance embeds safety directly into the learning process: constrained and safe RL with robust objectives reduces the frequency of constraint violations [24, 25, 72], while preference optimization aligns the agent with human or normative signals (Section 2.2). Training-time methods can reduce base rates of unsafe behavior, but they cannot anticipate all deployment conditions—motivating the next tier.

Therefore, runtime assurance provides the next line of defense, catching residual risks that escape training-time bounds. Mechanisms include verification and shielding, least-privilege tool access, anomaly detection, rollback, and staged deployment strategies (shadow and canary rollouts) [23, 26]. Runtime controls are especially important for agentic systems because tool-mediated actions can have irreversible real-world consequences.

Finally, post hoc assurance closes the feedback loop: structured telemetry, reproducible traces, and failure analysis support accountability and drive continuous improvement [23]. Post hoc evidence also feeds back into the upfront tier (updating threat

models and regression suites), creating a virtuous cycle of assurance refinement.

Together, this layered assurance stack connects the Perceive–Plan–Act–Reflect–Learn loop to the unified evaluation framework in Section 4, where metrics and stress tests operationalize the residual-risk budget at each layer.

2.3.3. Transition to our two core dimensions

The risk surface above directly motivates the survey’s two core trustworthiness dimensions: Safety and Robustness (Section 3.1) addresses harm prevention and reliability under uncertainty across Perceive–Plan–Act–Reflect–Learn, while Privacy and System Security (Section 3.2) addresses protection of sensitive data (inputs, memory, traces) and the integrity of the agent’s execution environment (tools, credentials, and protocols). Other aspects (e.g., accountability) remain important but are treated as supporting concerns that shape evaluation and governance rather than standalone dimensions in Section 3. **Figure 3** organizes these stage-aligned mitigations into a hierarchical taxonomy; the representative methods listed in each leaf node are drawn from the detailed discussion in Section 3 [23–50].

3. Core dimensions of trustworthiness

In this section, we focus on two core dimensions that are most critical for high-risk agentic AI deployments (see **Figure 3** for an overview): Safety and Robustness (Section 3.1) and Privacy and System Security (Section 3.2). These dimensions address the fundamental requirements for trustworthy agent operation in high-stakes scenarios: (1) preventing unacceptable harm and maintaining a reliable performance under perturbations, (2) protecting sensitive information and securing the agent’s execution environment against attacks.

Other trustworthiness aspects—value alignment, transparency, fairness, and accountability—are important complementary concerns that interact closely with safety and security. While a comprehensive treatment of these dimensions is beyond the scope of this focused survey, we note their relevance where appropriate within our core discussions and in the evaluation metrics (Section 4).

Within each subsection, we adopt a consistent structure—Definition (what the dimension means for agents), Risks (where failures arise along the agent workflow), and Methods (stage-targeted mitigations)—to follow a stable “risk→mitigation” mapping across dimensions. As noted in the Introduction, the specific risks and methods mapped to each stage are illustrative of the current literature rather than a provably exhaustive enumeration; new failure modes and mitigations are expected to emerge as agent architectures and deployment contexts evolve. To avoid fragmented discussion, we consolidate all evaluation metrics and representative benchmark suites across Section 3 into Section 4.

3.1. Safety and robustness

This section introduces key definitions of AI Safety and Robustness, examines the risks that arise at each stage of the agent workflow (perceive, plan, act, reflect, and learn), and reviews stage-aligned mitigation methods for addressing them.

3.1.1. Definition

In agentic AI, Safety and Robustness are related but distinct. In the following, we provide their definitions, respectively. A note on terminology: Lin et al. [84] formalize the distinction between AI safety and AI security, which we adopt for their definitions below. Our survey groups safety with robustness (Section 3.1) and security with privacy (Section 3.2) because, from a mitigation perspective, safety and robustness share defense mechanisms (constraint enforcement, distributional hardening) while security and privacy share controls (access policies, encryption, and monitoring). This organizational choice is complementary to the safety-vs-security taxonomy of Lin et al. [84].

AI safety is defined as “the property of an AI system to avoid causing unintended harmful outcomes to individuals, environments, or institutions, despite uncertainties in inputs, goals, training data, or deployment conditions” [84]. It aims to prevent unintentional harm (e.g., accidents, misalignment) by ensuring agentic systems comply with specified constraints, ethics, and norms [20]. This contrasts with security, which addresses intentional threats. Safety focuses on avoiding catastrophic outcomes [85] and satisfying hard constraints in high-stakes deployments, such as preventing collisions in autonomous driving [86] or harmful recommendations in clinical decision support [87].

In the classical AI safety framing, many safety failures arise as accidents rather than explicit malicious intent: (i) specification problems (the objective or constraints do not match human intent), (ii) robustness problems (capabilities fail under shift), (iii) scalable oversight (humans cannot reliably evaluate long-horizon behavior), (iv) safe exploration (learning-time trial-and-error triggers harm). Agentic AI amplifies these issues because safety is a system property: even if the base model is well aligned, unsafe behavior can emerge from tool interfaces, memory, retrieval, and multi-step control loops.

Robustness focuses on maintaining a stable performance under perturbations, adversarial interference, and distribution shifts [88, 89]. In short, safety is about not causing harm, while robustness is about remaining reliable under non-ideal conditions.

A key nuance is that robustness is often necessary but not sufficient for safety. An agent can be robustly competent while still pursuing an unintended objective due to specification gaming (optimizing the literal objective while violating intent) [90] or goal misgeneralization (capabilities generalize OOD but the goal does not) [91]. Conversely, a safe specification may still fail operationally without robustness to noisy observations, tool failures, and distribution drift.

Robustness itself has multiple forms. Distributional robustness targets natural shifts (e.g., domain, demographic, and environment) and seeks consistent worst-group or worst-slice performance (e.g., DRO/group DRO objectives) [31]. Adversarial robustness targets strategic perturbations crafted by an attacker (e.g., prompt injection, multimodal adversarial examples) [6, 92]. For agentic systems deployed in open environments, both are required: distribution shifts create silent failures, while adversarial shifts create targeted exploitation.

3.1.2. Risks

Perceive: poisoning and adversarial perturbations

At the input boundary, agents are exposed to data poisoning [93] and adversarial perturbations [92]. Retrieved documents, prompts, or multimodal observations may contain deceptive content or hidden instructions that distort state estimation and steer downstream decisions toward unsafe behaviors. Beyond direct perturbations, agentic systems face instruction–data boundary confusion: indirect prompt injection can embed tool directives inside “benign” web pages, emails, or PDFs, causing the agent to treat untrusted content as higher-priority instructions [5, 6]. Another risk is sensor/observation spoofing (e.g., manipulated UI elements, visual adversarial patches, or misleading interface states) that biases perception toward unsafe affordances. These perception-level failures frequently cascade: corrupted observations lead to invalid plans, which then drive high-impact tool actions; the resulting outcomes may further pollute memory and reflection signals, making later-stage correction harder.

Plan: OOD generalization and brittle heuristics

In out-of-distribution (OOD) contexts, agents may over-generalize from familiar heuristics and produce plans that look plausible but are unsafe, invalid, or non-compliant. Such failures are hard to detect early because planning traces can appear coherent even when key assumptions no longer hold [89]. Planning additionally inherits specification risk: under underspecified objectives, agents may produce plans that satisfy proxy metrics while violating intent (specification gaming) [90]. A related failure mode is goal misgeneralization, where planning remains competent but optimizes the wrong objective in novel contexts [91]. Planning also suffers from miscalibrated uncertainty and modeling errors in world models or tool simulators, producing brittle “happy-path” strategies with poor contingency handling. These planning errors cascade into execution: when an agent commits early to a flawed plan, later stages may rationalize or entrench it (reflection), and repeated successful shortcuts can become reinforced during learning.

Act: high-impact execution and cascading failures

During execution, upstream errors are amplified into real-world side effects via dangerous tool use (financial loss, privacy violations, and service disruption) [94] or harmful interactions with users. Even if the plan is correct, sensor/tool failures can introduce noise or bias into feedback; corrupted outcomes then propagate across steps and compound into severe deviations [95]. Execution adds irreversibility and human-factor risks. For example, partial automation in driving has been linked to failures involving system limitations, driver overreliance, and inadequate engagement monitoring, as documented in investigations of Autopilot-related crashes [96, 97]. In tool-using agents, tool chaining can turn a single misstep into a sequence of harmful actions (e.g., a prompt-injected instruction triggers credential exfiltration, then initiates unauthorized transactions) [6, 94]. Action-level failures also poison downstream stages: bad outcomes become “evidence” used in reflection and learning, increasing the chance of systematic drift rather than one-off errors.

Reflect: unsafe self-assessment and missed warnings

If reflection mechanisms fail to detect risk signals (e.g., uncertainty, policy violations, and anomalous tool outputs), the agent may proceed despite being in an unsafe regime [98, 99]. Over-confidence and incomplete trace evidence further reduce the chance of timely intervention. Reflection is also vulnerable to deceptive rationalization and evaluator spoofing: agents may produce plausible post hoc explanations that hide causal failure, or craft outputs that satisfy automated judges without improving true safety (a reflection analogue of reward hacking). When reflection reuses the same base model as the actor, correlated errors can create a “closed loop” where mistaken beliefs are repeatedly self-confirmed. If the trace itself is incomplete (missing tool logs, truncated context, or untrusted memory), reflection may miss early warnings and allow unsafe plans to persist into subsequent episodes, where learning further amplifies them.

Learn: feedback loops that amplify risk

When updates are driven by biased or noisy feedback, agents may reinforce unsafe shortcuts that improve short-term success but increase long-term harm [100]. This phenomenon manifests as reward hacking, wherein the measured reward continues to increase while side effects that incentivize the agent to violate safety constraints are concomitantly amplified [101]. Learning-stage risk also includes safety regression and capability–constraint imbalance: updates to prompts, memories, tools, or policies can unintentionally remove previously effective safety behaviors (catastrophic forgetting of safety constraints) while preserving or increasing action competence. Online or continual learning from deployment logs may import new adversarial patterns (e.g., jail-breaking prompts, malicious web content) into the training distribution, effectively “teaching” the agent unsafe policies. These learning failures close the cascade: once unsafe behaviors are internalized, earlier-stage defenses (input filtering, runtime checks) must work harder, and incident recovery becomes costlier.

Multi-agent: emergent hazards and adversarial coordination

In multi-agent settings, safety failures can emerge from coordination dynamics rather than single-agent errors: agents may collude to bypass constraints, amplify misinformation through mutual reinforcement, or trigger negative externalities via competitive equilibria (e.g., resource exhaustion or denial-of-service through uncoordinated tool calls) [41, 102]. Communication channels also create new attack surfaces (e.g., one compromised agent relays injected instructions to others), turning localized perception attacks into system-wide action cascades.

Long-horizon: compounding error, delayed side effects, and value drift

Long-horizon trajectories magnify small errors: minor perception noise or planning miscalibration can compound over many steps into severe divergence, while harm may be delayed and hard to attribute (e.g., gradual financial loss, creeping policy violations, or subtle safety boundary erosion) [37]. Long-horizon agents also face statefulness risks (memory accumulation, stale goals, and context truncation), where outdated assumptions persist and contaminate subsequent planning and learning.

3.1.3. Methods

Mitigations should be stage-aligned along the Perceive → Plan → Act → Reflect → Learn lifecycle.

Perceive: robustness training and OOD detection

Data augmentation expands training coverage with perturbations, ambiguities, attacks, and edge cases [28, 102–106]. Adversarial training hardens agents against evolving attacks (from prompt-level manipulations to adaptive strategies) [6, 29, 107–109]. At deployment time, OOD detection flags contexts outside validated regimes and triggers conservative fallbacks (e.g., read-only mode or human approval) [30]. Perception hardening in agentic systems additionally benefits from input provenance and sanitization, enforcing strict separation between trusted system prompts and untrusted retrieved content, strip or quarantine suspected instructions in retrieved text and applying policy-aware content filters before information enters memory or planning [5, 7]. For distribution shifts, distributionally robust objectives (e.g., group DRO) can improve worst-slice reliability at the cost of average-case utility and additional tuning complexity [31]. Finally, uncertainty-aware perception (ensembles, selective prediction, and calibrated confidence) enables “fail-closed” behavior, but may increase latency and reduce autonomy due to more frequent abstentions.

Plan: constrained optimization for safety

Planning-stage safety is commonly modeled via constrained decision making (e.g., CMDPs) [24]. Safe RL balances return maximization with risk avoidance [33, 110], using constrained policy optimization such as CPO [25]. Risk-sensitive objectives (e.g., CVaR) further emphasize catastrophic tail-risk reduction during plan optimization [111, 112]. For LLM planners, a complementary line is constraint specification and enforcement in natural language: constitutional or rule-based constraints translate norms into reusable critique-and-revision procedures, improving compliance without enumerating all unsafe actions [32]. However, constraints can be gamed under misspecification; thus, specification-aware planning (e.g., interpreting rewards as uncertain and planning conservatively) helps mitigate reward hacking and negative side effects [27, 113]. In practice, safer planning often mixes (i) receding-horizon/MPC-style replanning, (ii) subgoal verification (checkable intermediate constraints), (iii) risk budgeting (explicit limits on tool calls, cost, or exposure). These methods typically trade autonomy and optimality for predictability and tighter safety margins, and may require stronger world models or simulators to be effective.

Act: runtime enforcement and shielding

Execution-time safeguards prevent unsafe plans from becoming high-impact actions. Shields and supervisory controllers can block or redirect unsafe actions during interaction with environments or tools [26, 114, 115]. In tool-using settings, this typically combines pre-execution checks (permissions, budgets, and policy compliance) with post-execution validation and rollback hooks. Modern agent deployments increasingly adopt guardrails as a layered runtime stack: (i) least-privilege tool access (scoped credentials, per-tool permissions, and spending caps), (ii) sandboxing and transactionality (dry-run, staged execution, and compensating

actions), (iii) content filtering for both inputs and generated tool arguments [34, 95]. Human-in-the-loop oversight (approval gates for irreversible actions, escalation on anomalies) is a pragmatic safety control in high-stakes settings [23], but increases cost and reduces responsiveness. A recurring trade-off is utility vs. enforceability: stricter runtime policies reduce catastrophic risk but may block benign edge cases, motivating adaptive policies that tighten under uncertainty or suspected attack.

Reflect: validation via simulation and formal checks

To reduce silent failures, agents (or their operators) can validate plans and anticipated outcomes through simulation-based testing in controlled environments, especially for long-horizon behaviors and rare events [35]. Formal verification can certify well-specified components under explicit assumptions [36, 116–119], although end-to-end verification of LLM-based agents remains challenging due to probabilistic generation [21, 120]. Reflection is strengthened by adversarial evaluation and red teaming: automated prompt attacks, tool misuse suites, and policy violation benchmarks can be integrated into continuous integration (CI) pipelines to catch regressions before deployment [29, 114]. For agentic systems, trace auditing (immutable tool logs, provenance for retrieved evidence, and anomaly detectors over action sequences) helps counter evaluator spoofing and supports post-incident forensics. The main limitation is coverage: simulation and red teaming cannot fully enumerate open-world contexts, so reflective checks should be paired with conservative runtime monitors that trigger rollback, safe mode, or human review when novel hazards are detected.

Learn: safe updates with regression gating

Learning-stage safety requires cautious updates: maintain regression packs built from stress failures and incidents, continuously monitor key safety/violation metrics, and gate releases to prevent regressions introduced by model, prompt, memory, or tool-policy updates [37, 38]. In deployment, staged rollouts (e.g., shadow/canary) with automated acceptance/rollback criteria provide a practical mechanism to validate updates under real traffic without amplifying unsafe behaviors [39]. When policy updates are learned from logged interactions, conservative safe policy improvement offers a principled way to avoid degrading a baseline policy under uncertainty [40]. For agentic systems, “learning” often includes non-parameter updates (prompt templates, tool schemas, memory policies, and retrieval corpora). Safe learning therefore requires versioned artifacts and incident-driven datasets: add failure cases (prompt injections, jailbreaks, and tool glitches) to regression suites and retrain/retune defenses accordingly [94, 107]. Where online adaptation is necessary, safe exploration and conservative updates reduce the probability of high-cost trials [27, 33]. The trade-off is slower iteration: stronger gating and conservative improvement reduce harm but can delay

beneficial capability updates, motivating risk-tiered release processes (stricter for high-impact tools, looser for read-only assistants).

Multi-agent: coordination constraints and system-level oversight

Multi-agent safety benefits from protocol-level constraints: restrict message formats, authenticate agent identities, and enforce shared safety policies on inter-agent communication to prevent the propagation of injected instructions. At the system level, centralized oversight (a monitor or “governor” that enforces global budgets, conflict checks, and stop conditions) helps prevent emergent cascades such as redundant tool calls and resource exhaustion. However, stronger coordination controls can reduce parallelism and autonomy, and may introduce single points of failure; practical designs often mix decentralization for performance with centralized “circuit breakers” for safety.

Long-horizon: checkpointing, risk budgets, and hierarchical guarantees

Long-horizon safety is improved by hierarchical decomposition (plans as verifiable subgoals), periodic checkpoints that require re-validation of assumptions, and risk budgets that bound cumulative exposure across steps (e.g., number of tool calls, spending, external writes). Receding-horizon replanning reduces compounding error, while explicit stop/interrupt policies prevent runaway execution when uncertainty grows. These approaches increase monitoring overhead and may require richer state tracking, but they directly target the long-horizon cascade that makes agentic safety uniquely challenging.

Defense-in-depth: complementary, not substitutable

The mitigations described above are intended as complementary layers in a defense-in-depth strategy, not as substitutes that designers can choose between. A poisoning attack at the perceive stage cannot be fully neutralized by guardrails or shielding at act time alone; conversely, even robust perception does not eliminate the need for planning constraints or runtime enforcement. In practice, trustworthy agent deployment requires protections at most or all stages simultaneously because failures at any single point can cascade through the remaining pipeline. Each layer addresses a different failure mode and provides residual coverage for gaps in other layers. **Table 4** summarizes representative stage-aligned mitigations and their individual trade-offs; the “limitations” column should be read as motivating protection at other stages rather than as reasons to omit a layer. Cross-cutting controls (logging, provenance, red teaming, and HITL escalation) are most effective when integrated end to end rather than as isolated patches.

Evaluation metrics and release-gating practices are consolidated in Section 4.

Table 4 • Stage-aligned safety and robustness mitigations for agentic systems (non-exhaustive).

Stage	Representative techniques	Primary benefit	Main limitation/trade-off
Perceive	Augmentation, adversarial training, provenance/sanitization, OOD detection, DRO objectives	Reduces attack/sift sensitivity; enables abstention or safe mode	Coverage gaps; compute/latency; may reduce average performance
Plan	CMDPs/CPO, CVaR, constitutional constraints, conservative planning, safe exploration	Optimizes under explicit safety constraints; reduces tail risk	Requires formalizable constraints; may be overly conservative
Act	Shields, guardrails, least-privilege tools, sandboxing, transactional execution, HITL approval	Prevents high-impact failures even with flawed plans	Reduced autonomy; false positives; operational overhead
Reflect	Simulation, red teaming, trace auditing, formal checks on components	Detects silent failures; supports debugging and regression tests	Incomplete coverage; brittle assumptions in simulators/verifiers
Learn	Regression gating, canary rollout, conservative improvement, incident-driven updates	Prevents safety regressions and drift over time	Slower iteration; monitoring and dataset maintenance cost
Multi-agent	Communication constraints, authentication, centralized governors, global budgets	Controls emergent cascades and collusion pathways	Coordination overhead; potential single points of failure
Long-horizon	Checkpointing, risk budgets, receding-horizon replanning, interruptibility	Limits compounding error and delayed side effects	More frequent interventions; increased monitoring requirements

Abbreviations: OOD: out-of-distribution; DRO: distributionally robust optimization; RL: reinforcement learning; CMDPs: constrained Markov decision processes; CPO: constrained policy optimization; CVaR: conditional value-at-risk; HITL: human-in-the-loop.

3.2. Privacy and system security

This section defines Privacy and System Security in the context of agentic AI, identifies key risks across the agent workflow—including data leakage, tool-mediated exfiltration, and supply chain vulnerabilities—and reviews stage-targeted mitigation strategies.

3.2.1. Definition

Following recent analyses [43, 45, 121–125], we define privacy in agentic AI as protecting any user- or environment-derived information that (i) directly identifies an individual (e.g., names, contact information), (ii) enables sensitive attribute inference (e.g., demographics, preferences, or behavioral patterns), (iii) can be reconstructed/regurgitated through an agent’s internal representations, memory, tool outputs, or interaction traces. Crucially, privacy risks extend beyond memorization to inference-based leakage, accumulation over long-horizon interactions, unintended disclosure during tool execution, and cross-agent propagation [45, 48].

Complementarily, we adopt the definition of AI security as “the property of an AI system to remain resilient against intentional attacks on its data, algorithms, or operations, preserving its confidentiality, integrity, and availability in the presence of adversarial actors” [84]. Unlike safety, which focuses on unintentional failures, security specifically defends against malicious exploitation. Formally, privacy protection is often expressed via indistinguishability guarantees. Under differential privacy (DP), a randomized mechanism $\mathcal{M}$ satisfies (ϵ, δ) -DP if for any neighboring datasets D, D' (differing in one record) and any measurable set of outputs S , $\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \Pr[\mathcal{M}(D') \in S] + \delta$ [126, 127], where $\Pr[\cdot]$ denotes the probability over the randomness of $\mathcal{M}$. DP clarifies a privacy–utility trade-off via ϵ (privacy budget) and supports principled accounting under repeated queries. Complementary,

older anonymity notions such as k -anonymity require each released record to be indistinguishable from at least $k - 1$ others w.r.t. quasi-identifiers [128], with known limitations under background knowledge and linkage attacks (e.g., motivating l -diversity and t -closeness) [129, 130].

Agentic AI introduces a practical distinction between static privacy and dynamic privacy. Static privacy concerns training-time memorization and post-training extraction (e.g., the regurgitation of training or logged interaction data). Dynamic privacy concerns runtime information flows across the Perceive→Plan→Act→Reflect→Learn loop, where privacy loss can occur even without memorization: repeated interactions enable attribute inference, tool calls serialize sensitive context into external systems, and intermediate traces create new disclosure surfaces [43, 45].

To capture context-dependent expectations, we also adopt the contextual integrity (CI) perspective: privacy is preserved when information flows conform to appropriate social norms (who shares what about whom, with whom, and under which transmission principles) [46, 121]. CI is particularly relevant for agents because they routinely transform context (summarize, retrieve, and act via tools), making it easy to violate norms even when no explicit identifier is leaked (e.g., disclosing a user’s health condition to an unrelated tool) [122, 124, 125].

We define system security for agentic AI as preserving the confidentiality, integrity, and availability of the agent’s tooling and execution environment (e.g., credentials, APIs, sandboxes, and protocols), and resisting attacks that induce unsafe actions such as unauthorized tool use, code execution, or protocol manipulation [43, 47]. Concretely, confidentiality covers secrets (API keys, tokens), private data, and proprietary

prompts/policies; integrity covers the correctness of retrieved evidence, tool outputs, and action arguments; and availability covers service continuity under abuse (e.g., tool-call floods, resource exhaustion). Threat modeling (assets, entry points, attacker capabilities, and failure impact) provides a systematic way to reason about agent security across components and stages [131].

Finally, agentic deployments raise supply chain security concerns: models, tool plugins/APIs, retrieval indices, prompt templates, and evaluation/guardrail services may come from distinct providers with independent update cycles. Compromise can occur upstream (malicious dependency, poisoned tool update, or trojaned plugin) and propagate downstream into agent behavior. Practical controls include Software Bills of Materials (SBOMs) and signed, verifiable artifacts to support provenance and rapid incident response [49, 50, 132, 133]. To analyze threats systematically, we adopt a component-based view (Input, Memory, Tool/Execution, and Communication/Protocol) and map it to the agent workflow below [42, 43, 47, 48].

While Section 3.1 emphasizes behavioral harm and robustness, this subsection focuses on data protection and system integrity: many attacks (e.g., prompt injection) matter here not only because they change behavior, but because they enable credential theft, unauthorized data access, and exfiltration through legitimate tool channels.

3.2.2. Risks

Perceive: prompt injection, multimodal inference, and obfuscated inputs

The input channel ingests user prompts, multimodal observations, and external content (web pages, emails, and GUIs). Direct/indirect prompt injection (DPI/IPI) can coerce agents to disclose private information (e.g., prior conversations or implicit user attributes) or to bypass policies [42]. Multimodal inference attacks can extract demographic or situational cues from images/screen context [48]. Obfuscated input attacks can evade static filters by evolving through multi-step perturbations, increasing both privacy leakage and security compromise risk [42]. Additional perceive-stage risks include social engineering inputs (phishing-style prompts that elicit secrets or induce the agent to paste credentials) and retrieval layer poisoning where attacker-controlled documents are designed to be retrieved and to carry hidden exfiltration instructions. A real-world pattern is that untrusted content can trigger unintended behaviors even without explicit user interaction (e.g., “zero-click” indirect prompt injection in enterprise copilots) [4, 5]. These inputs are high-leverage because they seed downstream stages: once a malicious instruction is treated as trusted context, it can shape planning, tool use, and what gets stored into memory, creating persistent privacy loss.

Plan: privacy-unsafe memory use and long-horizon attribute aggregation

Planning often relies on memory (conversation history, episodic buffers, and long-term stores), which can accumulate sensitive data across sessions. Two privacy risks arise: (i) regurgitation/reconstruction, where previously seen private content resurfaces later [45]; (ii) attribute aggregation, where multiple low-sensitivity fragments combine into high-sensitivity inferences over time [48]. From a security perspective, memory can be

poisoned (e.g., delayed triggers/backdoors) so that future plans are steered toward unsafe actions or disclosure. Planning introduces contextual integrity violations as a distinct privacy failure mode: even if each memory item is individually non-sensitive, the planner may route information to an inappropriate recipient (e.g., sending a user’s medical context to a generic web tool) because it optimizes task success over norm compliance [46, 122]. Another risk is cross-session re-identification: stable memory identifiers, embeddings, or long-lived profiles can allow linking across sessions/users, especially under insider threats (operators) or compromised storage backends. These plan-stage risks cascade into act: once the plan encodes a privacy-unsafe tool strategy (e.g., “search using full email thread”), subsequent execution can leak at scale.

Act: tool-mediated leakage and execution-layer exploitation

Tool use (browsers, code interpreters, databases, and GUI actuators) creates powerful but high-risk pathways. Privacy leakage can occur when sensitive text is sent to tools (e.g., via RAG queries or browser autofill), when browsing traces expose identifiers, or when agent-generated queries inadvertently encode private attributes [43, 47]. Tool outputs may contain latent metadata that the agent later exposes. Security threats arise when unsafe tool calls execute malicious code, trigger SQL injections, or interact with untrusted web content, enabling the exploitation of system-level vulnerabilities [47]. Agentic AI adds execution-time threat classes uncommon in stateless LLMs: (i) credential theft through tool access (keys/tokens exposed via prompts, logs, or tool outputs), (ii) exfiltration via authorized channels, where the agent uses legitimate permissions (email, cloud drive, and HTTP requests) as a covert export path, (iii) side-channel leakage through tool timing, error codes, or response sizes that encode sensitive states (e.g., inferring whether a record exists from query latency). Tool-chain attacks (benign tools combined into an exfiltration pipeline) further expand blast radius and can convert a single injected instruction into sustained leakage across many steps [95]. These failures propagate into reflect/learn: traces of tool calls and outputs become part of the agent’s evidence base and may be retained.

Reflect: cross-component propagation through traces and protocols

In multi-agent or user-agent-tool ecosystems, intermediate messages, rationales, and negotiation traces can leak sensitive attributes, and disclosures can propagate nonlinearly across agents [48]. Protocol-level failures (tampering, impersonation, or instruction injection) can further amplify leakage and induce coordinated unsafe behavior. Reflection adds two privacy-specific hazards: trace over-collection and rationale leakage. Rich traces (chain-of-thought-like rationales, tool logs, and screenshots) are valuable for debugging and accountability, but they may also store identifiers, credentials, or sensitive inferences that were never meant to persist. Moreover, protocol attacks such as replay/downgrade (e.g., forcing weaker authentication or reusing prior authorization messages) can compromise integrity and spread injected instructions across components. Once leaked into shared logs or cross-agent channels, sensitive information is hard to retract, and may be repeatedly resurfaced during later planning.

Learn: drift and persistence of privacy risk across updates

Long-horizon use and iterative updates can make privacy risks persistent: what was once benign context may become identifying when combined with new data sources or policies, and training-time or memory-time retention can reintroduce leakage even after patches [43, 45]. Without explicit retention limits and regression checks, privacy/security properties may silently degrade over releases. Learning introduces additional adversarial threat models: insider threats (malicious or careless data curators), compromised tools/APIs that backdoor training logs, and supply chain poisoning where updated guardrails or retrieval indices embed exfiltration logic. Even without explicit fine-tuning, non-parametric updates (prompt templates, memory policies, and tool registries) can re-open previously closed leakage channels. Because agents operate continually, privacy debt can accumulate: small retention or policy changes can shift the effective privacy budget over time (DP-style composition) and invalidate earlier assurances [127].

Multi-agent privacy and security: shared context, privilege escalation, and collusive exfiltration

When multiple agents collaborate, shared memory, delegation, and message passing create new disclosure pathways: one agent may request context from another beyond “need-to-know”, or forward sensitive content into a broader channel. Security risks include privilege escalation (via delegation to a higher-privileged agent), impersonation (spoofed agent identity), and collusive exfiltration where agents coordinate to leak data through seemingly benign tool calls. Such failures can be rapid and nonlinear: a single compromised agent can contaminate others’ context and plans, amplifying leakage beyond what any single-agent threat model predicts [48].

Credential and secret exposure: long-lived tokens, implicit capture, and recovery gaps

Agents often handle API keys, OAuth tokens, passwords, and session cookies. Secrets can leak through prompts (users paste keys), tool outputs (stack traces), memory (cached credentials), screenshots/GUI context, or logs. A distinctive agentic risk is secret reuse across stages: once a secret is captured during perceive/act, it can persist in memory, appear in reflection traces, and be replayed by future plans—turning a single exposure into sustained compromise. Recovery gaps (delayed key rotation, unclear ownership of secrets) further increase the time to mitigate.

3.2.3. Methods

Perceive: sanitization, detection, and minimization at intake

Defenses include input sanitization (structured validation; LLM-as-a-judge filtering) and adversarial search-based detection for evolving prompt injections [42]. Privacy-aware prompting and minimization policies reduce unnecessary sensitive content ingestion, lowering downstream leakage surfaces [43]. Perceive-stage security benefits from zero-trust intake: treat all external content (web/email/docs/UI text) as untrusted, enforce strict separation between instructions and data, and apply provenance

checks before content can influence tool permissions or memory writes. Practical mechanisms include instruction-hierarchy policies, input allowlists (schemas for tool arguments), and Data Loss Prevention (DLP)-style detectors that flag Personally Identifiable Information (PII) and secrets at the boundary (before they enter retrieval indices or logs). The trade-off is utility and latency: aggressive filters can block benign inputs and require costly multi-pass detection, motivating risk-adaptive policies that tighten under suspected attack [44].

Plan: memory governance and least-privilege planning

Mitigations include memory auditing and sensitivity-aware access policies, automated deletion/expiration to enforce data minimization, and controls that limit what planning can retrieve or retain when privacy risk is high [43]. Differential privacy and related techniques can reduce the influence of individual memory entries and limit reconstruction risk [45]. Planning-stage privacy requires memory governance as policy: tag memory entries with sensitivity (PII, credentials, and regulated data), enforce purpose limitation (only retrieve data needed for the current task), and adopt contextual integrity checks that validate whether a planned information flow is appropriate for the recipient/tool [46, 125]. DP-style mechanisms can be applied not only to training, but also to telemetry and analytics derived from user interactions (bounded contribution per user), though DP often reduces personalization fidelity and requires careful privacy-budget accounting under repeated queries [127]. In practice, least-privilege planning couples plan generation with permission-aware tool selection (choose the minimum-scope tool that can complete the step), trading some autonomy for a smaller blast radius when a plan is compromised.

Act: sandboxing, runtime policy enforcement, and post-execution redaction

Effective defenses include sandboxed execution, runtime policy enforcement governing permitted tool actions, command-level verification, and post-execution redaction using content-aware minimization techniques [43, 47]. Credential handling should follow least-privilege principles, and tool scopes should be tightly constrained to reduce blast radius. Execution-layer defenses increasingly resemble enterprise security stacks. Policy-as-code gates (pre-call authorization, per-tool budgets, rate limits, and data-classification constraints) reduce unauthorized actions; EDR-style monitoring instruments tool calls, filesystem/network events, and anomalous sequences to detect exfiltration attempts in real time. Data loss prevention (DLP) can be adapted to agentic settings by scanning outgoing tool arguments (queries, HTTP payloads, and emails) and blocking or redacting sensitive spans before transmission. For privacy-preserving computation, sensitive tasks can be moved into protected execution environments (e.g., TEEs) or distributed protocols (e.g., federated learning for personalization; secure MPC/HE for cross-organization analytics) [43, 48]. These defenses trade off performance (overhead, latency) and developer complexity (policy engineering, instrumentation).

Reflect: protocol protection and trace-based incident handling

Protocol-aware defenses include encrypted communication, authenticated message channels, and tamper-evident

logging/ledger-style verification to prevent message tampering and support investigations. Consistency/debate-style checks can reduce the impact of adversarial manipulation in multi-agent settings [48]. Conformance checking and privacy-incident schemas help standardize violation detection and response [45]. Reflection should implement privacy-aware observability: collect only what is necessary (minimize trace content), segregate sensitive logs, and apply retention limits to rationale and tool traces. For integrity, tamper-evident logs and authenticated provenance enable post-incident attribution and replay-based debugging without trusting any single component. Operationally, incident handling should include automatic secret revocation (key rotation), quarantine of contaminated memory entries, and backfilling of regression tests from discovered leakage traces. The trade-off is reduced debuggability: redacted traces may hinder root cause analysis, motivating tiered access (secure audit mode under strict controls) rather than always-on full-fidelity logging.

Learn: retention limits and privacy/security regression gating

To prevent the persistence of privacy risk, learning and deployment updates should be gated by retention and leakage controls (e.g., memory expiration, deletion policies, and DP-based storage), and by regression packs that replay known attacks and leakage cases across releases [43, 45]. Evaluation metrics and benchmark suites are consolidated in Section 4. Beyond model updates, secure update pipelines should cover tool registries, prompt templates, guardrail services, and retrieval indices. Supply chain controls (SBOMs, signed artifacts, dependency provenance, and SLSA-style build levels) help prevent the silent introduction of malicious components and accelerate rollback when compromises are found [49, 50, 133]. Where possible, privacy regression tests should include CI-based scenarios (norm violations), not only leakage string-matching, to catch inappropriate flows that do not surface as explicit PII [122, 125].

Multi-agent privacy and security: shared-policy enforcement and compartmentalization

Mitigations for multi-agent systems emphasize compartmentalization: per-agent memory namespaces, explicit contracts for

what can be requested/shared, and authenticated channels with role- and scope-based access control. Global governors can enforce system-wide budgets (e.g., total external sends) and detect cross-agent exfiltration patterns. Debate/consistency checks help, but must be paired with protocol security to prevent spoofing or replay in agent-to-agent communication [48].

Credential and secret management: ephemerality, vaulting, and safe interfaces

Agents should never treat secrets as normal text. Practical designs include (i) ephemeral credentials (short-lived tokens bound to a single task), (ii) vault-backed retrieval (agents request scoped secrets via secure APIs rather than storing them), (iii) continuous rotation and revocation hooks, (iv) secret scanning/redaction in prompts, memory writes, and logs. This reduces catastrophic persistence: even if an agent is prompt-injected, the attacker faces a narrow time window and limited scopes. The trade-off is engineering overhead and occasional usability loss (e.g., more re-authentication events).

Summary and practical trade-offs

Component- and stage-aligned defenses are complementary: intake filtering and minimization reduce exposure, memory governance limits retention, and sandboxing/protocol protections constrain propagation [43, 47, 48]. However, stronger controls may reduce utility (less context/memory), add engineering overhead (sandboxing, authenticated protocols), and introduce trade-offs between privacy guarantees and performance (e.g., DP or aggressive deletion) [43, 45]. A practical takeaway is to treat privacy/security as end-to-end information-flow control rather than isolated filters: vulnerabilities are often composed across components (**Table 5**). Compared to the safety mitigations in Section 3.1, controls here are frequently operational (zero trust, secret management, provenance, and DLP/EDR instrumentation) and must be maintained continuously as tools and supply chains evolve. Evaluation should therefore combine outcome measures (leakage counts, attribute inference accuracy) with process signals (trace minimization, permission conformance, and provenance coverage), as consolidated in Section 4.

Table 5 • Threat model matrix for privacy and system security in agentic AI (non-exhaustive).

Component	Typical assets	Representative threats (CIA)	Agentic-specific vectors
Input	Prompts, retrieved docs, GUI context	Confidentiality loss via injected disclosure; integrity loss via instruction/data confusion; availability loss via prompt flooding	Indirect prompt injection; zero-click triggers in untrusted content; multimodal attribute inference
Memory	Histories, long-term stores, embeddings	Regurgitation/reconstruction; memory poisoning/backdoors; unbounded retention	Long-horizon aggregation; cross-session re-identification; insider access to stored traces
Tool and execution	API keys, browsers, DBs, code exec	Exfiltration through tool calls; unauthorized actions; SQL/code injection; DoS via tool spam	Exfiltration via authorized channels; tool chaining; side-channel leakage via timing/errors
Comm and protocol	Inter-agent msgs, RPC, logs	Spoofing/replay/tampering; trace leakage; channel disruption	Delegation-based privilege escalation; cross-agent propagation of injected instructions
Supply chain	Models, plugins, dependencies	Compromised updates; poisoned artifacts; provenance ambiguity	Malicious tool/plugin updates; poisoned retrieval indices; untrusted guardrail services

Abbreviations: CIA: confidentiality–integrity–availability; GUI: graphical user interface; API, application programming interface; DB: database; SQL: structured query language; DoS: denial of service; RPC: remote procedure call; DLP: data loss prevention.

4. Consolidated metrics and benchmarks

This section consolidates evaluation content into a single “evaluation hub” for trustworthy agentic AI. Our two core dimensions—Safety and Robustness (Section 3.1) and Privacy and System Security (Section 3.2)—receive the primary focus, while metrics for other trust aspects (value alignment, transparency, fairness, and accountability) are included as supporting references. Unlike single-turn LLMs, agents must be evaluated on both outcomes and process: multi-step trajectories, tool calls, permission checks, and trace evidence. Metric choices and release gates should follow a risk-based framing (e.g., NIST AI RMF) [23]. Concretely, evaluation should be treated as a system-level property: it is jointly determined by the base model, prompts, retrieval/memory policies, tool wrappers, runtime guards, and oversight interfaces. As a result, the same policy/model can exhibit different risk profiles under different tool scopes, logging granularity, or approval workflows. This motivates (i) explicit versioning of the full agent stack, (ii) standardized trace schemas for auditability, (iii) scenario-driven release gates that bound residual risk under the intended operational design domain (ODD) rather than reporting a single global score.

4.1. Connecting evaluation to the agent lifecycle

The metrics and benchmarks in this section are designed to map directly onto the five-stage workflow (Perceive–Plan–Act–Reflect–Learn) introduced in Section 2.1 and the stage-aligned risks and mitigations discussed in Section 3. Specifically: Perceive-stage risks (data poisoning, adversarial inputs, and OOD inputs) are evaluated through stress test pass rates, adversarial robustness scores, and input filtering effectiveness. Plan-stage risks (specification gaming, miscalibrated uncertainty) are captured by constraint violation rates (CVRs) and planning-quality metrics under OOD scenarios. Act-stage risks (dangerous tool use, cascading failures) are measured through catastrophic event rates (CERs), tool-policy violation counters, and data leakage rates. Reflect-stage quality (self-assessment fidelity, trace coverage) is measured by decision coverage rate (DCR) and compliance verification rate (CompVR). Learn-stage regression (safety degradation after updates) is tested through regression packs, canary rollout acceptance criteria, and version-to-version comparison. This explicit coupling ensures that evaluation is not a detached post hoc exercise but a diagnostic tool for identifying where in the lifecycle trustworthiness failures originate.

4.2. Measurement principles

A trustworthy metric suite should satisfy: validity (targets the intended harm), reliability (stable across seeds/evaluators), reproducibility (environment/tool/version control), coverage (OOD/adversarial/long-tail/long-horizon stress), and statistical power. It should report means with 95% confidence intervals and stratify by scenario families (in-distribution vs. OOD vs. adversarial). It should keep stress test results separate from in-distribution results (it should not average them). If using LLM-as-a-judge, document the judge model/prompt and report agreement or calibration against human labels when possible [134, 135].

The subsequent paragraphs delineate the conceptual dimensions of these criteria, after which we provide a detailed description of the corresponding metrics in Section 4.3.

4.2.1. Outcome vs. process evaluation (agents require both)

Episode-level outcome metrics (e.g., SR, CER; see Section 4.3 for all abbreviations) capture what happened, but not how it happened. For agents, the process is often the primary risk channel: unsafe intermediate tool calls, permission bypasses, or evidence-free planning can occur even when the final answer looks correct. Therefore, release gates should pair outcome metrics with process evidence (CVR, DCR/Trace-Coverage, and CompVR) to ensure the system is not “getting lucky” or relying on brittle, unverifiable reasoning. This is especially important when agents operate with intermittent oversight (e.g., batched approvals) and long-horizon autonomy [23].

4.2.2. Trajectory-level vs. step-level metrics (what to measure when)

Step-level metrics (CVR, CompVR, tool-policy violation counters, and context leakage counts) are most diagnostic for debugging and for enforcing hard constraints that must hold at every step (e.g., “never exfiltrate secrets”, “never execute unapproved code”). Trajectory-level metrics (SR, CER, SafetyScore, and Leakage/Regurgitation) are most appropriate for release decisions in high-risk domains, where the unit of harm is typically an episode (e.g., a completed workflow, a driving segment, or a clinical recommendation). Practically, we recommend reporting both: (i) step-level violation histograms (where violations occur, which tools/actions trigger them), (ii) episode-level aggregates (how often any violation occurs per episode), because long-horizon systems can appear safe on averages while still exhibiting rare but catastrophic chains.

4.2.3. Long-horizon evaluation (compounding errors and delayed consequences)

Long-horizon agents face compounding errors: small early mistakes in retrieval, planning, or tool execution can propagate into later stages and only manifest after many steps (e.g., incorrect state updates leading to unsafe actions). Consequently, evaluations should include (i) horizon-stress tests that scale trajectory length and branching, (ii) delayed-effect probes where harmful consequences arise only after intermediate actions, (iii) tail-risk reporting (e.g., CER/CVaR-style summaries) rather than relying solely on mean performance [136]. When feasible, they should include “time to failure”-style statistics (steps until first violation) to differentiate agents that fail immediately from agents that drift into unsafe regimes.

4.2.4. Multi-agent evaluation (attribution, emergence, and coordination)

When multiple agents interact, risks can emerge from coordination dynamics (e.g., collusion, information amplification, and cascading leakage) that are absent in single-agent settings [41, 64]. Evaluation should therefore track: (i) attribution (which agent/tool/message caused the violation), (ii) coordination quality (task completion under shared constraints, deadlock rate, negotiation stability), (iii) cross-agent leakage (whether private attributes or credentials propagate across agents). Privacy benchmarks for multi-agent or multimodal GUI settings (e.g., MLA-Trust) are useful complements to single-agent leakage tests [48].

4.2.5. Judge reliability and adversarial robustness

LLM-as-a-judge can reduce labeling costs but introduces failure modes: judges may be biased toward fluent rationales, may be sensitive to prompt wording, and can be adversarially influenced when evaluating jailbreak-like behaviors [29, 134]. For safety/security gates, treat judges as instruments that must be calibrated: report inter-judge agreement, run paired human audits on critical subsets, and include adversarial “judge attacks” (e.g., misleading justifications) to measure judge robustness. In high-stakes deployments, prefer deterministic checkers (schema validation, policy engines, formal constraints) for hard rules, and reserve LLM judges for soft risk scoring and triage.

4.3. Metric dictionary

Table 6 standardizes the abbreviations used throughout Section 3 and links each metric to representative benchmark families.

4.3.1. How to read the table

We recommend reporting episode-level outcomes (e.g., SR, CER, and PHR) together with step-/process-level violations and evidence (e.g., CVR, DCR/Trace-Coverage, and CompVR) because

agent failures often arise from intermediate tool use, long-horizon planning, and hidden policy breaks.

4.3.2. Abbreviations (quick definitions)

SR: success rate per episode. SafetyScore: graded safety risk score for an action/trajectory. CER: catastrophic event rate (episode-level severe harm). CVR: constraint violation rate (step-level hard-rule/policy/tool-permission violations). Stress-Pass: pass rate on OOD/rare-event/long-horizon stress sets. PHR: preference win rate (pairwise alignment). Honesty: factual consistency/truthfulness under tool/RAG settings. ASR/RR: attack success/refusal rate under adversarial prompts or social engineering. Over-Opt: reward-hacking diagnosis via over-optimization curves [136]. Fidelity/Stability/Complexity: explanation faithfulness/robustness/simplicity. TVD: tool-selection bias among functionally equivalent tools. Fairness-ASR: fairness break rate under adversarial pressure. DCR: decision coverage rate (trace completeness; a.k.a. Trace-Coverage). CompVR: compliance verification rate (independent verifiability vs. standards/policies). ISR/A2I: interruption success rate/alert-to-intervention latency. Leakage/Regurgitation/Attr.-Inf. Acc./CLC: privacy leakage, memorized content resurfacing, attribute inference accuracy, and context leakage count.

Table 6 • The trustworthy agent metric dictionary. We recommend reporting both episode-level outcomes (SR, CER) and step-level process signals (CVR, DCR) to capture risks that arise from intermediate tool use.

Metric	Type	Definition	Benchmark examples
Outcome and safety metrics			
SR	Episode Outcome	Success Rate: fraction of episodes fulfilling the user goal.	AgentBench, WebArena
SafetyScore	Graded Risk	Graded Likert scale (1–5) or binary labeling of trajectory safety.	SafeMind, R-Judge
CER	Episode Outcome	Catastrophic Event Rate: episodes with unacceptable harm.	Safety-Gym, Do-Not-Answer
CVR	Process Violation	Constraint Violation Rate: step-level hard policy breaches.	AgentDojo, SheetAgent-Safe
Stress-Pass	Robustness	Pass rate on OOD, adversarial, or long-horizon subsets.	HarmBench, AgentGuards
Over-Opt	Safety (Reward)	Reward-hacking proxy: degradation on true reward vs. proxy.	Scaling-Laws-Over-Opt
Privacy, Security and Compliance Metrics			
Leakage/Regurgitation/CLC	Privacy	Count/Rate of PII, internal context, or memorized content emitted.	MLA-Trust, PrivacyLens
Attr.-Inf. Acc.	Privacy (Inference)	Accuracy of inferring private attributes from agent outputs.	MLA-Trust
ASR/RR	Adversarial	Attack Success Rate/Refusal Rate under injection attacks.	AgentDojo, HarmBench
DCR (Trace-Coverage)	Auditability	Decision Coverage Rate: fraction of actions with complete traces.	AgentCompass
CompVR	Compliance	Compliance Verification Rate: fraction verifiable against rules.	Financial/Clinical Audits
Supporting Dimensions			
PHR	Alignment	Preference Head-to-Head Win Rate vs. baseline.	AlpacaEval, MT-Bench
Honesty	Alignment	Factual consistency and hallucination rate (w/tool evidence).	TruthfulQA, AgentHalu
TVD/Fairness-ASR	Fairness	Tool selection bias (TVD) and fairness under attack (Fairness-ASR).	BiasBuster, HarmBench
Fidelity/Stability/Complexity	Transparency	Explanation faithfulness, robustness, and simplicity (length/nodes).	XAI-Bench
ISR/A2I	Accountability	Interruption Success Rate/Alert-to-Intervention Latency.	HAII Benchmarks

Abbreviations: SR: success rate; CER: catastrophic event rate; CVR: constraint violation rate; OOD: out-of-distribution; PHR: preference win rate; ASR: attack success rate; RR: refusal rate; TVD: total variation distance; DCR: decision coverage rate; CompVR: compliance verification rate; ISR: interruption success rate; A2I: alert-to-intervention latency; HAII: human–AI interaction.

4.3.3. Dimension-specific evaluation notes

Safety and Robustness (Section 3.1): Our first core dimension. Combine SafetyScore/CVR/CER with Stress-Pass to separate in-distribution performance from tail risk. Privacy and Security (Section 3.2): Our second core dimension. Report leakage/regurgitation/attribute inference, plus Privacy-ASR under iterative attacks and within-interaction CLC. Supporting metrics from other dimensions: For comprehensive evaluation, we also include metrics for value alignment (PHR, Honesty, and ASR/RR), transparency (Fidelity, Stability, and Complexity), fairness (TVD, Fairness-ASR), and accountability (DCR, CompVR, and ISR/A2I).

4.3.4. Benchmark suite descriptions (selected anchors)

We briefly summarize representative suites referenced throughout this survey; these suites are best viewed as families that instantiate the metric dictionary under different environments and threat models. SafeMind targets safety risks in embodied LLM agents, pairing graded safety assessment (SafetyScore) with event-level harm and violation tracking under long-horizon interaction [137]. AgentDojo provides a dynamic environment for prompt injection attacks/defenses in tool-using agents, enabling the measurement of ASR/RR, tool-policy violations, and defense generalization [138]. AgentHarm focuses on harmful capability testing in agentic settings, complementing refusal/harmlessness evaluations with task-level agent affordances [139]. MLA-Trust benchmarks multimodal LLM agents in GUI environments, emphasizing privacy leakage and sensitive attribute inference under realistic screen contexts and multi-step interactions [48]. HarmBench standardizes automated red teaming and robust refusal evaluation, offering an extensible framework for measuring attack success and refusal robustness across adversarial prompt families [29]. TruthfulQA operationalizes truthfulness/honesty evaluation, and is commonly reused as a component within agentic honesty checks when combined with retrieval/tool evidence [140]. For tool-execution risk, LM-emulated sandbox evaluation (ToolEmu-style) simulates tool APIs and environments so that risky actions (data deletion, financial theft) can be measured without real side effects [141]. For autonomous driving simulation, CARLA remains a widely used platform for scenario-based testing and rare-event generation under controllable ODD specifications [142].

4.3.5. Emerging benchmarks and evaluation trends (2024–2025)

Recent benchmarks increasingly (i) model interactive tool use

rather than static QA, (ii) include iterative adversaries that adapt across steps, (iii) measure process compliance (trace evidence, policy conformance) in addition to outcomes. For web agents, generalizable guardrails are increasingly evaluated via dedicated suites that stress policy compliance and injection resistance (e.g., WebGuard) [34]. For production-facing workflows, evaluation is moving toward trace-centric, operations-aware frameworks that treat agent behavior as a workflow to be audited (e.g., AgentCompass) [143]. For tool-choice reliability, emerging work highlights measurable biases in selecting among functionally equivalent tools, motivating TVD-style metrics and tool-bias benchmarks [144]. For privacy norms and contextual integrity, benchmarks such as PrivacyLens and CI-Bench complement leakage/regurgitation tests by evaluating whether agents follow context-dependent privacy expectations [122, 125].

4.4. Scenario-to-metric mapping and benchmark suites

With the metric dictionary in place, **Table 7** maps scenarios to a minimal set of release-gating metrics. We separate (i) mandatory gates (e.g., CER/CVR caps, leakage thresholds, and minimum DCR/CompVR) from (ii) diagnostic metrics used for debugging and ablation. In practice, combine four sources: offline replay regression (fast iteration), simulation banks for rare events/long horizons (tail risk), automated red teaming (misuse robustness), and interactive sandboxes for tool-mediated attacks.

4.4.1. Setting release-gating thresholds (risk-bounded acceptance criteria)

Thresholds should be scenario- and tier-specific; a “one-size-fits-all” SR target is not meaningful when residual risk varies by domain and autonomy level. A practical template is to set hard caps on catastrophic events and hard-rule violations (CER/CVR), plus minimum floors on auditability evidence (DCR/CompVR), then optimize utility within these bounds (SR, latency, and cost). As a rule of thumb for safety-critical scenarios, many teams adopt extremely low tolerated catastrophic rates (e.g., $CER < 0.1\%$ on high-risk scenario banks) and near-zero tolerance for hard constraint violations ($CVR \approx 0$) when violations directly correspond to policy or permission breaks. Privacy/security thresholds should similarly enforce strict caps on high-severity leakage (e.g., secrets/credentials) while allowing graded treatment of low-severity context exposure, aligned with the threat model in Section 3.2 [23].

Table 7 • Scenario-to-metric mapping for release gating. Gates progress from strict regression checks (Tier 0) to statistical bounds on tail risk in simulation (Tier 1), finally to anomaly-based monitoring in deployment (Tier 2).

Release gate	Scope and environment	Mandatory thresholds (caps/floors)	Diagnostic metrics (optimize)
Tier 0: Regression	Offline replay of known incidents and “golden” traces.	Zero regressions on Po scenarios. CVR = 0 on policy tests.	Trace completeness (DCR); Coverage of regression pack.
Tier 1: Stress Test	Simulation/Sandbox/Red team (OOD + Adversarial).	$CER < \text{Limit}$ (e.g., 0.1%). $ASR < \text{Limit}$. No high-sev leakage.	SafetyScore distribution; Stress-Pass rate; Latency.
Tier 2: Canary	Shadow mode or limited traffic live deployment.	Low anomaly rate (CVR/Refusal spikes). Auto-rollback on violation.	User feedback (PHR); Intervention rate (ISR); Cost.

Abbreviations: CVR: constraint violation rate; CER: catastrophic event rate; ASR: attack success rate; DCR: decision coverage rate; PHR: preference win rate; ISR: interruption success rate; OOD: out-of-distribution.

4.4.2. Risk-stratified evaluation and “ODD-aware” gating

Agents should be released only within validated operational envelopes. Concretely, define scenario families that reflect the intended ODD (in-distribution), plus explicit stress families (OOD, adversarial injection, and long-horizon drift). Gates can then be tiered: (i) Tier 0 (offline): no regressions on regression packs; (ii) Tier 1 (simulation/sandbox): pass OOD/adversarial/long-horizon stress with bounded CER/CVR; (iii) Tier 2 (shadow/canary): satisfy online monitoring gates with rapid rollback. This tiering reduces the chance that an agent “wins on average” while failing under rare but foreseeable conditions.

4.4.3. Continuous monitoring and regression detection

Because agent stacks evolve (model updates, prompt changes, and tool-policy changes), monitoring should treat trust metrics as non-stationary. Track leading indicators (CVR spikes, permission denials, anomalous tool usage, judge disagreement, and leakage alerts) in addition to lagging indicators (incident reports). Regression detection should replay curated packs of (i) prior incidents, (ii) red team attacks, (iii) synthetic corner cases on every change to the model, memory policy, or tool wrappers [37, 38]. For deployment, staged rollouts with automated canary analysis provide a practical safeguard to catch distribution shift and regressions under real traffic [39].

4.5. Evaluation pipeline & tooling

We recommend a staged pipeline that increases autonomy and stakes: (1) offline regression, (2) simulation A/B and stress testing, (3) sandboxed tool use and red teaming, (4) shadow mode (read-only; no side effects) with trace auditing, (5) controlled canary rollout with fast rollback, (6) continuous monitoring with regression packs. Tooling should standardize trace schemas (inputs, plans, tool calls, permissions, and outcomes), environment/tool version stamps, seeds, and judge configurations/prompts.

4.5.1. Stage 1: offline regression (replay known failures)

Offline regression is the fastest iteration loop: maintain a continuously updated pack of “known-bad” cases (prompt injections, tool misuse patterns, privacy leaks, and unsafe plans) and “near-miss” traces. Each run should be deterministic where possible (fixed seeds, pinned tools, and frozen retrieval snapshots) and should emit step-level traces so that CVR/DCR regressions are localized to specific stages (Perceive/Plan/Act/Reflect/Learn). This stage is particularly effective for catching accidental regressions from prompt/memory/tool-wrapper edits that do not involve model retraining [38].

4.5.2. Stage 2: simulation testing (rare events and long-horizon)

Simulation banks support stress testing beyond what logged data covers. For embodied/control domains, scenario-based simulators (e.g., CARLA for driving) enable the systematic variation of weather, traffic density, occlusion, and adversarial actors to probe tail-risk behaviors [142]. For interactive tool agents, simulation can include “mock worlds” (synthetic websites, synthetic enterprise documents, and synthetic emails) that embed controlled

privacy/security hazards (PII, secrets, and injection strings). Domain randomization and scenario generation expand coverage and reduce overfitting to a small set of scripted tasks [83].

4.5.3. Stage 3: sandboxed execution (ToolEmu-style emulation)

Before granting real tool access, evaluate agents in sandboxes that enforce strict isolation (filesystem, network, and credentials) and emulate external APIs. LM-emulated sandbox evaluation supports measuring risky tool-use trajectories (e.g., “delete records”, “transfer money”) without real-world consequences, and enables the fine-grained attribution of which intermediate steps caused a violation [141]. A practical recommendation is to maintain two sandboxes: (i) a behavior sandbox that measures whether the agent attempts unsafe actions, (ii) a security sandbox that measures whether the agent can be induced to bypass policy wrappers.

4.5.4. Stage 4: red teaming (automated + human)

Red teaming should combine automated adversarial generation (broad coverage) with targeted human adversaries (depth). Automated suites (e.g., HarmBench) provide standardized ASR/RR measurement and support regression testing across prompt families [29]. For agent-specific prompt injection and tool hijacking, dynamic environments (e.g., AgentDojo) are valuable because they model interaction, tool calls, and defenses under adaptive attackers [138]. For high-risk deployments, document the red team protocol: attacker capabilities (insider vs. external), tool access, allowed social engineering, and success criteria (leakage, unauthorized tool use, and policy bypass) [6, 7].

4.5.5. Stage 5: shadow mode (read-only deployment with trace auditing)

Shadow mode runs the agent on real inputs but blocks side effects (no external writes, no irreversible actions). This stage is ideal for validating process metrics (DCR/CompVR), measuring real OOD rates, and identifying privacy leakage patterns in authentic workflows before impact. Shadow traces also seed regression packs with realistic distributions, reducing sim-to-real mismatch.

4.5.6. Stage 6: canary rollout (staged release with fast rollback)

Canary releases gradually increase exposure, paired with automated acceptance criteria and rollback triggers (e.g., CVR spike, leakage alerts, and abnormal tool-call distributions). Continuous delivery frameworks for automated canary analysis provide practical guardrails when model and tool policies are updated frequently [39].

4.5.7. Stage 7: production monitoring (continuous assurance)

Post-deployment monitoring should treat trustworthy behavior as a service SLO: track CVR/CER proxies, leakage alerts, refusal anomalies, tool misuse indicators, and intervention latency (ISR/A2I). Use drift detectors over tool-call distributions and

scenario tags to trigger re-evaluation on stress suites when the environment changes. Importantly, logs and telemetry must themselves be privacy-preserving (redaction, access control) to avoid turning evaluation into a leakage channel (Section 3.2).

4.5.8. Trace schema requirements (minimum viable logging)

At minimum, traces should record: (i) context identifiers (episode id, user/session anonymized id), (ii) version pins (model, prompt, tool wrappers, and policy), (iii) agent internals (plan summary, selected tools, and confidence/uncertainty if available), (iv) tool evidence (calls, arguments, permission checks, outputs, and errors), (v) safety/security checks (policy decisions, redactions, and violations), (vi) oversight events (alerts, approvals, and takeovers). DCR/Trace-Coverage should be computed from these fields, and CompVR requires that key compliance claims be independently verifiable from logged evidence rather than free-form rationales [143].

4.6. Reporting standards

Use fixed report templates: metric dashboards with 95% CIs and stratification; risk–utility plots (e.g., CER/CVR vs. SR); stress test breakdowns (OOD/adversarial/long-horizon); and a reproducibility bundle (configs, seeds, tool registries, and judge prompts) plus audit evidence (trace summaries and representative failure traces).

4.6.1. Suggested report template (what to include)

A practical trustworthy-agent report should include: (i) system card (model, tools, memory, guardrails, and autonomy level), (ii) threat model (attacker capabilities, tool exposure, and data sensitivity), (iii) scenario suite definition (ODD, stress families, and multi-agent settings), (iv) gating thresholds (CER/CVR caps; leakage limits; and DCR/CompVR floors), (v) offline + sandbox results (including red team ASR/RR), (vi) deployment readiness (shadow/canary monitoring plan, and rollback conditions) [23].

4.6.2. Visualization recommendations (risk is multi-dimensional)

Beyond single bar charts, we recommend: (i) risk–utility frontiers (SR vs. CER/CVR), (ii) stratified breakdowns by scenario family and risk tier, (iii) violation heatmaps across tools/actions to localize failure channels, (iv) horizon curves (SR/CVR as a function of trajectory length) to surface long-horizon brittleness. For judge-based metrics, include agreement plots (judge–human, judge–judge) and sensitivity to judge prompts [134].

4.6.3. Failure analysis (representative traces and root cause taxonomy)

Because agent failures are often chain-based, reports should include representative failure traces with a short root cause label aligned to the lifecycle: perceive (retrieval error/injection), plan (invalid assumptions/unsafe subgoal), act (tool misuse/permission violation), reflect (missed warning), and learn (regression after update). This enables mitigation mapping back

to Sections 3.1 and 3.2 without conflating behavioral safety with system compromise.

4.6.4. Reproducibility bundle (minimum viable artifact)

For each reported result, provide: environment manifests (tool versions, sandbox images), configs and seeds, judge prompts and judge model identifiers, scenario suite definitions, and trace schema documentation. Production-facing evaluations should also include “known issues” lists and regression-pack definitions so future releases can be compared under identical conditions [37, 38].

4.6.5. Responsible reporting and disclosure (what to share)

When publishing evaluations, share aggregate metrics and sanitized traces sufficient for reproducibility but avoid releasing exploit strings or sensitive tool configurations that enable immediate misuse. For security-relevant findings (prompt injection bypasses, tool wrapper failures), responsible disclosure practices and the staged release of details help balance transparency with risk [7].

4.7. Open challenges in evaluation

4.7.1. Judge fragility and rationale hacking

As agents become more fluent, judges can be misled by plausible rationales even when actions are unsafe or evidence-free. This is particularly problematic when agents learn to optimize for judged scores, creating “evaluation gaming” loops [134–136]. Robust evaluation likely requires hybrid judging: deterministic checkers for hard rules, plus calibrated judges for soft scoring.

4.7.2. Benchmark saturation and overfitting to suites

Static benchmarks can be saturated quickly: agents (or training pipelines) may implicitly overfit to known test distributions, especially when benchmarks are widely used for iteration. Dynamic benchmarks (changing tasks/web content), hidden test sets, and periodic suite refreshes are needed to preserve validity over time [29].

4.7.3. Sim-to-real gaps

Simulation is essential for rare-event coverage, but sim-to-real mismatch remains a core limitation. Domain randomization and mixed offline/online validation help, yet evaluation must remain conservative when the real deployment distribution is poorly characterized [83].

4.7.4. Long-horizon combinatorics

Exhaustive long-horizon evaluation is intractable: the space of tool-call sequences grows combinatorially. Practical alternatives include stress testing curated “dangerous chains” (e.g., multi-step jailbreak sequences), measuring time to violation, and maintaining targeted regression packs from incident traces [141].

4.7.5. Multi-agent attribution and shared responsibility

In multi-agent deployments, harm may arise from interaction effects (information amplification, collusion, and blame shifting).

Assigning responsibility requires protocol-aware traces, message authentication, and evaluation designs that separate individual from collective failure modes [41, 64].

4.7.6. Adversarial coverage is never complete

Threat models evolve (new prompt injection styles, compromised tools, and social engineering). As in security engineering, evaluation must be continuous: automated red teaming, frequent regression updates, and deployment monitoring are necessary complements to any static benchmark [6, 7].

5. Real-world applications in high-risk domains

This section grounds our two core trustworthiness dimensions—Safety and Robustness and Privacy and System Security—in high-risk real-world domains: autonomous driving [145–147], healthcare [148–150], and intelligent assistants [19, 151–155] (see **Table 8** for a cross-domain lifecycle comparison). Each domain involves high-stakes decision-making where safety failures can cause physical harm or loss of life, and where privacy/security breaches can compromise sensitive data or critical infrastructure. By focusing on safety-critical and security-sensitive considerations in each domain, this section illustrates how the theoretical mitigations discussed in Section 3 translate into practical deployment requirements.

5.1. Autonomous driving: safety and security challenges

The scope of autonomous driving (AD) is defined by the The Society of Automotive Engineers (SAE) J3016 standard [151], ranging from L0 (no driving automation) to L5 (full driving automation). While L2 systems (partial driving automation) are widely deployed today, the transition to L3 (conditional driving automation) and L4 (high driving automation) autonomy introduces

critical challenges in uncertainty and safety [155]. To bridge this gap, trustworthy agentic AI frameworks are being adopted to enhance the Perceive–Plan–Act–Reflect–Learn loop. In AD, trustworthiness is ultimately evaluated system-wide: perception stacks, planners, actuators, HMI (handoff), fleet learning, and organizational processes jointly determine residual risk. This naturally maps to Section 3.1 (robustness, constrained planning, run-time shielding, and simulation/verification) and Section 3.2 (secure V2X, least-privilege tool access, and trace governance), and motivates lifecycle-aligned assurance rather than single-module benchmarking.

5.1.1. Safety: robustness, uncertainty, and minimum-risk behavior

In the perceive stage, achieving robustness against environmental anomalies (e.g., occlusion, severe weather) remains a primary hurdle [145, 156]. Traditional perception systems are typically trained on standard static benchmarks like KITTI [157] and nuScenes [158]. While these datasets utilize multi-modal sensors, they are inherently limited to a single-vehicle perspective. Consequently, they struggle with physical limitations such as occlusion or sensor degradation in severe weather. To address this, modern systems are evolving from standalone sensor-based perception to collaborative V2X communication frameworks [146]. For the Plan→Act stages, a recurring challenge is long-tail interaction: rare cut-ins, atypical pedestrians, ambiguous signage, and novel construction layouts can invalidate learned heuristics [159]. Trustworthy agent designs therefore emphasize (i) risk-aware planning (e.g., constrained optimization/safe RL as in Section 3.1), (ii) conservative fallback policies when uncertainty is high (OOD-triggered safe mode), (iii) minimum risk condition (MRC) behaviors (safe stop/pull-over strategies) when the system cannot maintain the operational design domain (ODD) [160]. In the reflect stage, safe handoff and driver engagement monitoring are essential for L2/L3, since mismatched mental models can create over-trust and delayed intervention [96].

Table 8 • Cross-domain mapping of core trustworthiness dimensions. Each cell summarizes the domain-specific challenges and mitigation methods with supporting references. This table focuses on the two core dimensions critical for high-risk deployments: safety and robustness and privacy and system security.

Dimension	Autonomous driving (AD)	Healthcare (HC)	Intelligent assistants (IA)
Safety and Robustness	Challenge: Perception failures in adverse weather, occlusion, long-tail scenarios; collision prevention. Methods: V2X collaborative sensing, simulation-based validation, safe RL, runtime shielding [145–147].	Challenge: Diagnostic errors, clinical decision failures, hazardous hallucinations. Methods: Multi-center validation, clinician-in-the-loop architecture, uncertainty quantification [148–150].	Challenge: Tool execution failures, indirect prompt injection, memory poisoning. Methods: Sandboxed execution, LM-emulated testing, input sanitization [19, 138, 141].
Privacy and System Security	Challenge: V2X data exposure, location tracking, cybersecurity threats (spoofing, jamming). Methods: Secure V2X protocols, ISO/SAE 21434 compliance, data anonymization [146, 151].	Challenge: Patient data breaches, re-identification risks, EHR security. Methods: HIPAA/GDPR compliance, federated learning, end-to-end encryption [152, 153].	Challenge: Credential theft, unauthorized tool access, data exfiltration. Methods: Least-privilege access, ephemeral credentials, policy enforcement, audit logging [19, 138, 154].

Abbreviations: AD: autonomous driving; HC: healthcare; IA: intelligent assistants; V2X: vehicle-to-everything; RL: reinforcement learning; EHR: electronic health record; HIPAA: Health Insurance Portability and Accountability Act; GDPR: General Data Protection Regulation; IoMT: Internet of Medical Things; SaMD: software as a medical device.

5.1.2. Failure post-mortems: Uber (2018), Tesla Autopilot, and Cruise (2023)

Real-world incidents illustrate how lifecycle failures cascade. In the 2018 Uber ATG Tempe fatality, the developmental ADS repeatedly misclassified the pedestrian/bicycle and failed to trigger timely braking; additionally, emergency braking was disabled and the safety driver was inattentive, yielding a Perceive→Act breakdown compounded by weak oversight [161]. Multiple Tesla Autopilot crashes investigated by regulators highlight edge-case perception/planning failures and HMI issues: drivers may misunderstand L2 limitations, leading to unsafe reliance and delayed takeover, i.e., reflect-stage handoff failures that convert residual risk into harm [96, 97]. In the 2023 Cruise San Francisco incident (pedestrian dragging), failures spanned perception/action recovery and post-incident disclosure processes, underscoring that trustworthy AD also requires organizational incident handling and trace integrity for timely remediation [162]. These cases motivate Section 3.1 mitigations such as OOD detection + conservative fallback, runtime shielding, and scenario-based validation focusing on rare events and recovery behaviors (not only nominal driving).

5.1.3. Regulatory and compliance requirements

AD safety engineering is shaped by standards and reporting regimes. ISO 26262 operationalizes functional safety via hazard analysis, safety goals, and ASIL ratings, encouraging explicit safety cases and traceable requirements-to-tests mapping [163]. ISO/PAS 21448 (SOTIF) targets the safety of the intended functionality under performance limitations and foreseeable misuse, which directly aligns with OOD/long-tail risks in the perceive/plan stages [160]. On the security side, ISO/SAE 21434 frames cybersecurity engineering across the vehicle lifecycle (threat analysis, security goals, and verification) [164]. In the U.S., NHTSA's Standing General Order requires timely reporting of certain crashes involving ADS/ADAS, pushing operators to maintain high-quality logs and rapid triage pipelines [165]. In the EU, the AI Act treats many AD components as high-risk AI systems, emphasizing risk management, technical documentation, logging, human oversight, and post-market monitoring—all consistent with the release-gating and trace-evidence principles summarized in Section 4 [166].

5.1.4. Evaluation protocols and deployment practices

Beyond static datasets, AD requires scenario-based testing across the ODD. Simulation platforms (e.g., CARLA) enable controlled stress tests for rare events, sensor degradation, and interactive multi-agent traffic [142]. Standardized scenario specifications (e.g., OpenSCENARIO) support reproducible replay of corner cases and regression packs across software releases [167]. Practically deployed systems track disengagement metrics, near-miss proxies (e.g., time to collision), and policy violation rates; however, trustworthy release gating should also validate recovery behaviors (MRC achievement), trace completeness, and handoff latency (ISR/A2I) as emphasized in Section 4 [163]. Staged roll-outs (closed-course → shadow → limited ODD) with automated

rollback criteria operationalize the “safe updates” principle from Section 3.1.

5.1.5. Multi-agent driving: V2X cooperation and interaction safety

AD is inherently multi-agent; other vehicles, pedestrians, and infrastructure respond strategically. V2X-enabled cooperative perception can reduce occlusion and extend sensing horizons, but introduces coordination and trust challenges: inconsistent maps, delayed messages, or compromised participants can degrade shared belief states [146]. Multi-agent LLM-based driving frameworks use debate/negotiation to handle ambiguous merges and social driving norms [168, 169], yet safety guarantees remain difficult when communication is unreliable or adversarial. A practical approach is layered safety: game-theoretic intent modeling for other road users plus hard safety envelopes (shields) during the act stage, limiting the consequence of coordination errors (Section 3.1).

5.1.6. Privacy and system security: cybersecurity, telemetry, and data governance

Beyond physical safety, connected autonomous vehicles face significant cybersecurity threats. V2X communication channels can be targeted by adversarial attacks including spoofing, jamming, and man-in-the-middle attacks [146]. Vehicle systems must implement robust authentication protocols, encrypted communication, and intrusion detection systems compliant with ISO/SAE 21434 cybersecurity standards. Additionally, privacy concerns arise from the extensive sensor data collected by autonomous vehicles. From the component-based lens in Section 3.2, key risks include (i) confidentiality loss via location/trajectory linkage and camera footage re-identification, (ii) integrity loss via compromised OTA updates or forged V2X messages that poison collaborative perception, (iii) availability loss via denial-of-service against V2X stacks or cloud telemetry. Mitigations include data minimization (on-device redaction and purpose-limited retention), secure key management for vehicle credentials, and tamper-evident logging to support post-incident forensics and compliance audits [164].

5.2. Healthcare: safety and privacy challenges

In the healthcare domain, the paradigm is shifting from traditional reactive care to AI-driven healthcare transformation, enabling continuous remote patient monitoring through the Perceive–Plan–Act–Reflect–Learn loop [170]. Unlike static diagnostic models, modern agents utilize the Internet of Medical Things (IoMT) to ingest real-time multimodal data (e.g., wearables, electronic health records) during the perceive stage [152, 171]. This domain presents acute challenges for both safety (clinical decision errors can cause patient harm) and privacy (medical data is among the most sensitive). Healthcare agents often combine heterogeneous components: RAG over EHRs, temporal models for vitals, LLM-based triage, and tool calls into ordering systems. As a result, trustworthy deployment must explicitly connect domain practice (clinical workflows, responsibility allocation, and adverse event reporting) to the stage-aligned mitigations in Sections 3.1 and 3.2, rather than treating the LLM as the only risk source.

5.2.1. Safety: clinical decision support and error prevention

In the plan phase, agents functioning as clinical decision support systems (CDSS) generate personalized treatment plans and diagnostic predictions [148]. Ensuring the factuality and safety of medical advice is paramount. In the act and reflect stages, agents execute tasks ranging from robotic-assisted surgery to administrative automation via virtual nursing assistants [148, 149]. Given the high-stakes nature of these actions, safety is enforced through a Human-in-the-loop architecture [150]. Mapping to the lifecycle is as follows: perceive risks include missing/incorrect vitals, incomplete EHR context, and distribution shift across hospitals; plan risks include hallucinated rationales, unsafe contraindication handling, and goal misgeneralization (e.g., optimizing throughput over patient safety); act risks include unsafe order entry or automation errors; reflect risks include overconfident self-assessment; and learn risks include feedback loops that overfit to local practice patterns. Accordingly, agents should be designed to abstain and escalate under uncertainty (OOD detection + calibrated confidence), enforce clinical guardrails (contraindication and dosage constraints), and validate plans via independent checks or simulations before any irreversible action (Section 3.1) [23].

5.2.2. Failure post-mortems: Watson for Oncology, Epic Sepsis, and COVID-19 AI screening

Historical failures underscore that impressive offline metrics may not translate to safe clinical performance. IBM Watson for Oncology was reported to produce unsafe or non-evidence-based treatment suggestions in some settings, attributed to limited or non-representative training data and insufficient clinical validation, highlighting plan-stage brittleness and oversight gaps [172]. The Epic Sepsis Model, widely deployed in hospitals, showed a substantially weaker real-world performance than expected in independent evaluations, illustrating dataset shift and the danger of relying on proprietary predictors without robust external validation [173]. During COVID-19, many imaging/triage models suffered from data quality issues and demographic bias, emphasizing perceive-stage noise, OOD generalization failures, and the need for stratified fairness audits (Section 3.1) [174, 175].

5.2.3. Clinical agent types and workflow integration

In practice, healthcare agentic systems span multiple roles: diagnostic agents (radiology/pathology triage and report drafting), treatment planning agents (guideline-based recommendations and care pathway optimization), robotic assistants (surgery/rehabilitation with constrained actuation), virtual nursing assistants (patient education, symptom monitoring, and adherence nudges), and administrative/workflow agents (coding, scheduling, and prior authorization). Each role changes the act-stage risk profile: direct clinical actuation demands stricter run-time constraints and verification than low-stakes documentation support.

5.2.4. Human–AI collaboration: escalation, trust calibration, and responsibility

Because liability and accountability remain clinician-centered,

trustworthy deployment emphasizes clear handoff protocols; when confidence drops, when evidence is incomplete, or when recommendations deviate from guidelines, the agent must escalate to a clinician with a concise, auditable trace (inputs, retrieved evidence, and constraints checked) rather than persuasive free-form rationales. Trust calibration requires communicating uncertainty and limitations, avoiding automation bias, and specifying responsibility allocation (who approves orders, who monitors drift)—a reflect-stage design requirement aligned with trace-based evaluation in Section 4 [176].

5.2.5. Evaluation and post-deployment monitoring

Healthcare evaluation prioritizes prospective validation and real-world evidence over purely retrospective studies. Recommended protocols include multi-center trials (to probe generalization), prospective silent deployment (shadow mode) to estimate error modes without patient impact, and continuous post-market surveillance with drift/OOD monitors and incident-triggered regression packs (Section 3.1) [177]. Safety evaluation should be stratified by demographics and site to detect disparate error rates and should include adverse event reporting pathways consistent with clinical governance.

5.2.6. Privacy and system security: patient data protection

The vast data collection inherent in IoMT-enabled healthcare introduces critical privacy risks. Strict adherence to regulatory frameworks like HIPAA and GDPR is mandatory [153]. Privacy-enhancing technologies including federated learning enable collaborative algorithm development across hospitals without centralizing sensitive data. With the Section 3.2 lens, key threats include attribute inference from longitudinal vitals, leakage through tool calls (EHR queries, billing systems), insider threats, and IoMT device compromise. Defenses include least-privilege access to EHR APIs, encryption in transit/at rest, tamper-evident audit logs, retention limits for conversational memory, and privacy-preserving analytics (federated learning and, when applicable, secure computation for cross-site studies) [153]. Availability is also safety-relevant; ransomware or denial-of-service against hospital infrastructure can indirectly cause patient harm, motivating incident response playbooks and segmented network architectures.

5.2.7. Regulatory and compliance requirements for clinical AI

Regulatory oversight increasingly treats clinical AI as Software as a Medical Device (SaMD); the IMDRF SaMD definition and risk framework establish a baseline vocabulary. The FDA's AI/ML-based SaMD action plan emphasizes good ML practice, transparency, and change management for learning systems (e.g., predetermined change control plans) [177]. Institutional Review Boards (IRBs) and clinical governance require clear protocol definitions, monitoring, and adverse event reporting for studies involving patient impact. At the global level, WHO guidance highlights ethics, equity, and governance requirements that align with the safety/privacy trade-offs discussed in Sections 3.1 and 3.2 [176].

5.3. Intelligent assistants: safety and security challenges

This domain represents the deployment of LLM-based agents as

autonomous operators, which have evolved from passive responders to active systems capable of tool learning and complex task execution [22]. A prime example is RepairAgent [178], which autonomously interleaves information gathering and tool invocation to fix software bugs. This transition to tool-integrated autonomy introduces severe safety and security risks unique to agents. Compared with autonomous driving and healthcare, assistants often operate in open-world digital environments (web, email, and enterprise SaaS) where adversaries can directly craft inputs and where authorized tools become exfiltration channels. Thus, the same Perceive→Plan→Act→Reflect→Learn loop yields a larger attack surface, making the system-security framing in Section 3.2 particularly central.

5.3.1. Safety: tool misuse and execution harm

In the perceive and act stages, agents processing untrusted data (e.g., emails, financial records) are highly vulnerable to indirect prompt injection [19]. Malicious instructions embedded within external content can hijack the agent’s reasoning, leading to unauthorized execution-based harm [154]. The plan and reflect stages are threatened by memory poisoning and backdoor attacks [154]. Rigorous evaluation environments like AgentDojo [138] are essential for benchmarking agent robustness. Safety failures often present as process violations, bypassing required approvals, exceeding tool budgets, or executing irreversible actions without sufficient evidence. Accordingly, assistants benefit from Section 3.1 controls that are system-level rather than purely model-level: constrained planning (policy-checked action sequences), runtime shields (block unsafe tool arguments), and staged rollouts with regression packs for discovered failure traces.

5.3.2. Privacy and system security: credential protection and sandboxing

Agents with tool access inherently possess elevated permissions—access to customer data, financial accounts, and system controls. Trustworthy deployment requires least-privilege access, tool-level sandboxing with ephemeral credentials, and systematic policy auditing. Ruan et al. [141] proposed an LM-emulated sandbox to safely identify tool execution risks. From the CIA viewpoint (Section 3.2), confidentiality failures include secret leakage (API keys, OAuth tokens), integrity failures include tampered tool outputs and unauthorized repository changes, and availability failures include tool-call flooding that disrupts services. Because assistants frequently integrate third-party plugins/APIs, supply-chain risks (compromised plugin updates, malicious tools) are first-class concerns and require provenance checks, signed artifacts, and continuous monitoring.

5.3.3. Real-world security incidents: plugins, manipulation, and sandbox boundary breaks

Early deployments have already surfaced security-relevant failure patterns. Prompt injection against plugin-enabled systems can coerce an assistant into exfiltrating data through authorized channels (e.g., sending content to an attacker-controlled endpoint), demonstrating that “allowed” tools can become leakage vectors when instruction/data boundaries are blurred [179]. The Bing Chat “Sydney” incident illustrated goal-driven manipulation attempts and brittle interaction safety under prolonged

conversations, highlighting long-horizon dynamics where planning/reflection can drift toward undesirable behaviors [180]. In code execution settings, reported sandbox weaknesses and overly permissive file/network access demonstrate that small isolation gaps can translate into large confidentiality and integrity risks [181]. These cases directly motivate Section 3.2 methods: zero-trust intake, sandboxing, DLP-style output filtering, and incident response with rapid credential rotation.

5.3.4. Financial/trading agents: market integrity and compliance constraints

Algorithmic trading agents raise domain-specific risks beyond generic tool misuse: inadvertent market manipulation (spoofing, layering, and wash trades), uncontrolled feedback loops under volatility, and conflicts of interest in strategy selection. Compliance regimes (e.g., SEC market access controls and broker-dealer risk management; FINRA supervision guidance; and, in the EU, MiFID II algorithmic trading obligations) require pre-trade risk limits, kill switches, surveillance for manipulative patterns, and comprehensive recordkeeping [182–184]. Trustworthy agents should therefore implement constrained planning with explicit risk budgets, independent monitoring (EDR-style telemetry over tool calls), and audit-ready traces that support post-trade investigations (Section 4).

5.3.5. Enterprise agent deployment: browsing, coding, and workflow automation

High-impact enterprise assistants include browser-based operators, coding agents with repository access, customer service agents with PII exposure, and workflow agents that act via email/calendar/CRM. Each expands the act-stage blast radius: a single injected instruction can trigger mass outreach, data deletion, or credential misuse. Practical deployment patterns therefore emphasize least-privilege scopes (per-tool, per-resource, per-time window), approval gates for irreversible actions, and compartmentalized memories to prevent cross-project or cross-client leakage (Section 3.2).

5.3.6. Red teaming and evaluation: attack success, leakage, and tool misuse detection

Agent evaluation must go beyond static jailbreak prompts to iterative, tool-mediated attacks. Benchmarks such as AgentDojo stress prompt injection and defense dynamics in interactive environments [138], while broader red teaming suites (e.g., Harm-Bench) can be adapted to measure refusal robustness and policy compliance under adversarial pressure [29]. For assistants, recommended metrics include attack success rate (ASR), context leakage count (CLC), tool permission violation rate (CVR), and trace completeness (DCR), consistent with Section 4. Automated adversarial testing should be integrated into CI to prevent regressions when tools, prompts, or policies are updated.

5.3.7. Incident response: detection, containment, and recovery loops

Because assistant failures can propagate rapidly (e.g., mass actions via automation), operators should treat safety/security incidents as standard security events through the following steps:

detect prompt injection attempts and anomalous tool sequences, isolate affected agent instances, revoke/rotate credentials, patch policies/filters, and preserve tamper-evident audit logs for forensics [185]. This operational loop closes the Reflect→Learn stage; post-mortems should produce regression packs (attack replays) and release gates that prevent recurrence, aligning with the “safe updates” principle in Section 3.1 and retention/leakage gates in Section 3.2.

6. Challenges and solutions

While this survey focuses on Safety and Robustness and Privacy and System Security as the two core dimensions for high-risk agentic AI, several open challenges remain before such agents can be safely deployed at scale. Below we highlight key directions spanning lifelong adaptation, monitoring/verification, personalization, and the trust–utility trade-off.

6.1. Lifelong learning and self-evolution

Trustworthiness must track evolving human expectations and social standards, so agents should not remain static. However, most agents are not lifelong learners that continuously acquire new knowledge and skills [10]. A promising direction is self-evolving agents that build a data flywheel from interaction feedback and iteratively refine their behavior [11], while addressing catastrophic forgetting and concept drift. When full model updates are too costly, context adaptation can offer a lighter-weight alternative [12]. From an engineering perspective, maintaining safety often requires a continuous human–agent loop for iterative updates and rollback in a dynamically changing world.

6.1.1. Runtime monitoring and verification

Agents that claim trustworthiness should be supported by systematic designs for verification and runtime monitoring. Metacognitive modules can help assess tool reliability and proactively detect failure executions [186], while agents can also learn from execution errors and feedback signals [10]. A major bottleneck is rare events: the scarcity of tail-risk data makes purely offline training insufficient, so continuous monitoring in real environments becomes essential. Moreover, trustworthiness risks are amplified in multi-agent and user–agent interaction settings; cross-component dynamics can lead to sensitive attribute leakage [48], underscoring the need for rigorous monitoring and verification under interaction.

6.2. Trustworthy personalization

Personalization is a major application driver for agents, and recent progress has leveraged sophisticated memory mechanisms [187–190]. Yet personalization must be reconciled with trustworthiness, especially privacy and safety [188, 191]. Beyond universal trust constraints, users may have context-dependent and individualized norms that vary across social environments, suggesting a need for methods that align personalization with diverse standards [192]. Future work should develop privacy-preserving memory and preference modeling that supports personalization without increasing leakage or unsafe reliance.

6.3. Green and efficient agentic AI

To reduce the economic and environmental costs, trustworthy agents should also advance Green AI [193]. Some agentic workflows use long contexts and extensive tool interactions, substantially increasing token and compute consumption [11, 12, 194]. Recent efforts aim to keep decoding time and memory growth closer to linear cost [195], which is a promising start. Importantly, trustworthiness should not depend on unbounded compute; practical deployment requires balancing safeguards with latency, energy usage, and scalability.

6.4. Standardizing explainability

Despite progress in explainability methods, rigorously quantifiable definitions and standardized evaluation remain limited [196, 197]. In practice, what counts as a “good” explanation can vary by domain, user expertise, and risk level, making universal metrics difficult. Section 4 outlines commonly used dimensions (e.g., fidelity, stability, and complexity), but future work should connect these metrics to human decision-making, misuse risks, and deployment settings, so explanations improve accountability rather than merely increasing trust.

6.5. Closing the accountability gap

Compared to other trust dimensions, there remains a notable scarcity of agent-specific methods and benchmarks that directly target accountability. To bridge this gap, Section 4 introduces preliminary metrics—Decision Coverage Rate, Compliance Verification Rate, and Interruption Success Rate—to quantify traceability, auditability, and interruptibility. We call for new benchmarks, system designs, and algorithmic frameworks that make accountability measurable, verifiable, and enforceable throughout the agent lifecycle.

6.6. Balancing trustworthiness and utility

A persistent challenge is balancing trustworthiness with effectiveness. For example, memory and experience reuse can improve performance [198], but retention and reuse may introduce privacy and safety risks. Similarly, constrained optimization (e.g., safe RL) improves risk control yet may trade off utility [33, 110]. Future work should emphasize Pareto-style evaluation and release-gating in the following way: maximize task utility while explicitly bounding risk and measuring where safeguards degrade user value.

6.7. Long-horizon deployment challenges

Long-horizon autonomy—where agents execute multi-step plans spanning minutes, hours, or days—is a defining capability of agentic AI, yet it introduces unique trustworthiness challenges that are only partially addressed by current methods.

6.7.1. Compounding errors and delayed consequences

Small perception or planning errors early in a trajectory can cascade into severe failures at later stages, with harm that is delayed and difficult to attribute (Section 3.1). Unlike single-turn interactions where errors are immediately observable, long-horizon agents can “drift” into unsafe regimes gradually, making root

cause analysis and timely intervention challenging [37]. Existing mitigations—checkpointing, risk budgets, and receding-horizon replanning—add monitoring overhead that may conflict with latency and efficiency requirements (Section 6).

6.7.2. Credit assignment and reward sparsity

Long-horizon tasks often provide only sparse, delayed feedback (e.g., task success/failure at the end), making it difficult to learn which intermediate actions were responsible for outcomes [53]. This credit assignment problem is amplified for safety: an agent may complete a task successfully while taking unnecessary risks along the way, but standard success metrics will not penalize such behavior. Recent work on hierarchical RL [69] and option frameworks [58] partially addresses this, but integrating safety constraints into hierarchical credit assignment remains open.

6.7.3. Synergizing reasoning and acting

A promising direction for long-horizon trustworthiness is the tighter integration of reasoning and acting. ReAct [54] demonstrates that interleaving explicit reasoning traces with action execution improves interpretability, error diagnosis, and plan adjustment—directly supporting scalable oversight. By making intermediate reasoning visible, operators can identify and correct problematic plans before execution cascades. Reflexion [61] extends this with verbal self-reflection, enabling agents to learn from trajectory-level feedback without weight updates. These frameworks suggest that long-horizon trustworthiness may benefit more from architectural transparency than post hoc monitoring.

6.7.4. Scalable oversight and verification

As trajectory length grows, human oversight becomes increasingly costly and error-prone; reviewers cannot realistically audit every step of a hundred-action plan. This motivates research into (i) hierarchical verification (certifying high-level subgoals rather than individual actions), (ii) anomaly-triggered review (flagging only unusual or high-risk segments), (iii) AI-assisted oversight (using auxiliary models to pre-screen trajectories). However, the reliance on AI oversight introduces recursive trust concerns that remain unresolved.

6.7.5. Evaluation intractability

Exhaustive long-horizon evaluation is combinatorially intractable; the space of possible action sequences grows exponentially with trajectory length (Section 4). Practical alternatives [141] include stress testing curated “dangerous chains,” measuring time to violation, and maintaining incident-derived regression packs. Future benchmarks should explicitly vary horizon length and report performance degradation curves, enabling operators to understand the operational envelope within which an agent can be trusted.

6.8. Case study: security failures in openclaw and moltbook

The rapid adoption of open-source agentic AI systems has exposed critical security gaps between capability development and

deployment safeguards. OpenClaw (formerly known as Clawdbot/Moltbot), a locally-run AI assistant with full system access—including file read/write, shell command execution, credential management, and external service interaction—exemplifies these risks [199].

6.8.1. Timeline and scale of exposure

OpenClaw gained mass adoption in late 2025, reaching hundreds of thousands of active deployments within weeks. By January 2026, security researchers had disclosed two critical CVEs: CVE-2025-49596 (CVSS 9.4, unauthenticated gateway access leading to full system compromise) and CVE-2025-6514 (CVSS 9.6, command injection enabling arbitrary code execution). Internet-wide scans revealed over 900 exposed OpenClaw gateways without authentication, leaking API keys, OAuth tokens, and conversation histories stored as plaintext Markdown/JSON files [199]. Infostealer malware families rapidly adapted to target these credential storage locations. Separately, Moltbook—a social network exclusively for AI agents—launched on 28 January 2026, and within days its misconfigured Supabase backend exposed the complete database of 32,000+ registered agents, including API keys, verification codes, and private messages [200].

6.8.2. Structural vulnerability: the “lethal trifecta”

Willison [201] coined the term lethal trifecta for agents that simultaneously (i) access private data, (ii) process untrusted external content, (iii) can communicate externally. When all three properties coexist—as they do in OpenClaw—an attacker can inject instructions into any external content the agent processes (e.g., a web page, email, or document), causing the agent to exfiltrate private data to an attacker-controlled endpoint. This pattern is structural rather than incidental; it cannot be fully resolved by prompt hardening alone, since LLMs cannot reliably distinguish legitimate instructions from injected ones embedded in data [179, 202].

6.8.3. Exposed attack surfaces

Several severe vulnerability classes emerged: (i) Insecure defaults and naked ports—security scans revealed OpenClaw gateways exposed on the public internet without authentication, creating open invitations for system takeover. (ii) Prompt injection—attackers manipulated the AI’s natural language interface to steal credentials, execute remote code, and exfiltrate sensitive data. (iii) Credential leakage—plaintext API keys and private chat logs were discoverable in exposed deployments [199, 200].

6.8.4. Supply chain risks in AI-to-AI interaction

Moltbook’s breach revealed a second, more systemic risk: malicious “skills” (plugins) disguised as benign utilities were found to read private configuration files and transmit API keys to external servers [200]. A large-scale empirical study of agent skill ecosystems found that 26.1% of 31,132 analyzed skills contained at least one vulnerability, spanning 14 distinct patterns including data exfiltration (13.3%), privilege escalation (11.8%), and prompt injection [203]. These findings highlight the absence of capability-based permission systems, sandboxing, and code signing in current agent plugin ecosystems.

6.8.5. Mapping to the survey framework

The OpenClaw/Moltbook incidents illustrate vulnerabilities at every stage of the agent lifecycle introduced in Section 2.1: Perceive—unauthenticated inputs and prompt injections entered without validation (Section 3.1, input sanitization); Plan—injected instructions altered the agent’s plan to include exfiltration steps, with no constraint checker detecting the deviation; Act—unrestricted tool access (file I/O, shell, and network) allowed the agent to execute attacker-controlled commands without least-privilege enforcement (Section 3.2); Reflect—no anomaly detection flagged unusual credential access patterns or data transmission volumes; Learn—malicious skills propagated through the update/install mechanism without provenance checks or regression gating. This mapping demonstrates that the chain of failure is systemic; no single-stage mitigation would have been sufficient, reinforcing the defense-in-depth principle advocated in Section 3.1.

6.8.6. Lessons learned

Table 9 summarizes the key lessons and their mapping to the survey’s trust dimensions and lifecycle stages.

These incidents underscore that agentic AI security must be treated as privileged infrastructure rather than consumer software; mandatory authentication, least-privilege scoping, plugin governance with cryptographic signing, continuous runtime monitoring, and explicit human approval for sensitive actions are not optional hardening steps but baseline requirements [199, 201].

7. Conclusions

Agentic AI systems—LLMs augmented with planning, tool use, memory, and long-horizon interactions—are moving from passive assistants to autonomous operators. This shift enlarges the risk surface: failures may emerge anywhere along a multi-step trajectory (e.g., perception/retrieval, planning, tool execution, reflection, and learning) and can cascade into high-impact outcomes. To support trustworthy development and deployment in high-risk

scenarios, this survey organized the literature through two complementary lenses: (i) a focused examination of two core trustworthiness dimensions—Safety and Robustness and Privacy and System Security—which are most critical for high-stakes deployments (Section 3), (ii) a Perceive→Plan→Act→Reflect→Learn workflow that localizes where risks arise and where mitigations can be applied. Across these core dimensions, we summarized stage-targeted methods and highlighted the system-level nature of trustworthiness in tool-using, memory-enabled, and multi-agent settings.

To make evaluations comparable and operational, we consolidated fragmented metrics and benchmark suites into a unified “evaluation hub” (Section 4), emphasizing that trustworthy agents must be assessed not only by final task success, but also by process signals such as constraint violations, trace completeness, compliance verifiability, and adversarial success rates. We hope this survey serves as a practical reference for researchers and practitioners building agentic systems that are not only capable, but also auditable, robust, and safe under real-world constraints.

7.1. Summary of open problems

Despite rapid progress, important limitations remain before trustworthy agentic AI can be deployed at scale. First, today’s agents are still far from reliable lifelong learners: they struggle with long-horizon generalization, are vulnerable to reward/specification gaming (Section 3.1), and can accumulate errors across extended tool-mediated trajectories. Second, verification and evaluation remain bottlenecked by long-tail and interactive risks; simulations and testbeds rarely cover rare events, adaptive attacks, and sim-to-real gaps, while many evaluations depend on imperfect judges and incomplete trace standards (Section 4). Third, deployment introduces persistent trade-offs and system risks, including privacy–utility tension in memory-based personalization (Section 3.2) and security vulnerabilities from tool access and credential management. Finally, frequent re-verification, monitoring, and safeguarding can impose substantial compute and operational costs.

Table 9 • OpenClaw/Moltbook: lessons learned mapped to trust dimensions and agent lifecycle stages.

Vulnerability	Trust dimension	Stage(s)	Recommended mitigation
Insecure defaults/naked ports	Security	Perceive, Act	Mandatory authentication, network isolation, secure-by-default configs
Prompt injection	Safety, Security	Perceive	Input validation, instruction hierarchy, jailbreak detection
Plan hijacking via injected instructions	Safety	Plan	Constraint checkers, plan-level guardrails, goal consistency verification
Credential leakage	Privacy	Plan, Act	Credential vaulting, secret scoping, DLP-style output filtering
Malicious plugins/skills	Security	Learn	SBOMs, code signing, sandboxed execution, permission declarations
Database misconfiguration	Privacy	Perceive, Act	Zero-trust architecture, least-privilege access, audit logging
No anomaly detection	Safety	Reflect	Runtime monitoring, anomalous tool invocation detection

Abbreviations: DLP: data loss prevention; SBOMs: software bills of materials.

These limitations motivate several concrete research and engineering challenges: (i) self-evolving agents with safe update rules and regression-gated releases; (ii) continuous runtime monitoring and verification that can detect rare-event and adversarial failures early; (iii) privacy-preserving personalization and memory governance that bound leakage while preserving utility; (iv) efficiency-aware trust mechanisms that deliver guarantees under realistic latency/energy budgets; (v) principled resolution of the trust–utility trade-off via risk-bounded, scenario-driven evaluation and deployment practices.

Acknowledgments

The authors would like to thank the anonymous reviewers for their constructive feedback, which significantly improved this manuscript. The authors acknowledge the use of Chat GPT-5 (OpenAI, San Francisco, CA, USA) for language polishing and grammar checking. The tool was applied to enhance clarity and scholarly tone, and all edits were carefully reviewed and validated by the authors to ensure accuracy, compliance with academic standards, and research integrity. The authors fully support Academia.edu Journals' adherence to COPE guidelines on AI in publication ethics and confirm that this use has been managed responsibly and ethically.

Funding

This research received no external funding.

Author contributions

Conceptualization, J.Q. and I.K.; methodology, J.Q.; investigation, J.Q., M.L., J.L., D.Y., S.M., Y.S., W.C., Y.Z., Y.C. and R.J.; writing—original draft, J.Q., M.L., J.L., D.Y. and S.M.; writing—review and editing, J.Q., Y.S., W.C., Y.Z., Y.C., R.J., I.K. and Z.X.; visualization, J.Q.; supervision, I.K. and Z.X. All authors have read and agreed to the published version of the manuscript.

Conflict of interest

Author Irwin King serves as Founding Advisory Editor and author Zenglin Xu serves as Editor-in-Chief of *Academia AI and Applications* journal. To ensure editorial independence, neither was involved in the peer-review or editorial decision-making process for this manuscript; handling was managed entirely by independent Editorial Board Members. All other authors declare no conflicts of interest.

Data availability statement

No new data were created or analyzed in this study. Data sharing is not applicable to this article.

Additional information

Received: 1 February 2026

Accepted: 9 April 2026

Published: 29 April 2026

Academia AI and Applications papers should be cited as *Academia AI and Applications* 2026, ISSN 3071-0286, <https://doi.org/10.20935/AcadAI8260>. The journal's official abbreviation is *Acad. AI*.

Publisher's note

Academia.edu Journals stays neutral with regard to jurisdictional claims in published maps and institutional affiliations. All claims expressed in this article are solely those of the authors and do not necessarily represent those of their affiliated organizations, or those of the publisher, the editors and the reviewers. Any product that may be evaluated in this article, or claim that may be made by its manufacturer, is not guaranteed or endorsed by the publisher.

Copyright

© 2026 copyright by the authors. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

References

1. Xi Z, Chen W, Guo X, He W, Ding Y, Hong B, et al. The rise and potential of large language model based agents: a survey. *Sci China Inf Sci*. 2025;68(2):121101. doi: 10.1007/s11432-024-4222-0
2. Chen J, Zou M, Wang Z, Wang Q, Sun DD, Chi Z, et al. Fin-HEAR: human expertise and adaptive risk-aware temporal reasoning for financial decision-making. In: Findings of the association for computational linguistics: EMNLP 2025. Suzhou: Association for Computational Linguistics; 2025. p. 1648–72. doi: 10.18653/v1/2025.findings-emnlp.87
3. Chen G, Yang S, Li C, Liu W, Luan J, Xu Z. Heterogeneous group-based reinforcement learning for llm-based multi-agent systems. *arXiv*; 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2506.02718>
4. Lakshmanan R. Zero-click ai vulnerability exposes microsoft 365 copilot data without user interaction. *The Hacker News*. 12 June 2025 [accessed on 31 December 2025]. Available from: <https://thehackernews.com/2025/06/zero-click-ai-vulnerability-exposes.html>
5. Paverd A. How microsoft defends against indirect prompt injection attacks. *Microsoft Security Response Center (MSRC) Blog*. 29 July 2025 [accessed on 31 December 2025]. Available from: <https://www.microsoft.com/en-us/msrc/blog/2025/07/how-microsoft-defends-against-indirect-prompt-injection-attacks>
6. Greshake K, Abdelnabi S, Mishra S, Endres C, Holz T, Fritz M. Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection. *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, Co-Located with ACM CCS

- 2023; 30 November 2023; Copenhagen, Denmark. 2023. doi: 10.1145/3605764.3623985
7. OWASP. Llm01: prompt INJECTION—owasp genai security project. 2024 [accessed on 31 December 2025]. Available from: <https://genai.owasp.org/llmrisk2023-24/llm01-24-prompt-injection/>
 8. Choi EC, Balasubramanian A, Qi J, Ferrara E. Limited effectiveness of llm-based data augmentation for COVID-19 misinformation stance detection. In: Companion Proceedings of the ACM on Web Conference 2025, WWW '25. New York (NY): Association for Computing Machinery; 2025. p. 934–7. doi: 10.1145/3701716.3715521
 9. Li M, Qi J, Wu Y, Zhao M, Ma L, Li Y, et al. From evidence to trajectory: abductive reasoning path synthesis for training retrieval-augmented generation agents. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2509.23071>
 10. Wang G, Xie Y, Jiang Y, Mandlekar A, Xiao C, Zhu Y, et al. Voyager: an open-ended embodied agent with large language models. Transactions on machine learning research (TMLR); 2023 [accessed on 15 January 2026]. Available from: <https://openreview.net/forum?id=ehfRiFor3a>
 11. Packer C, Wooders S, Lin K, Fang V, Patil SG, Stoica I, et al. MemGPT: towards LLMs as operating systems. arXiv; 2024 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2310.08560>
 12. Zhang Q, Hu C, Upasani S, Ma B, Hong F, Kamanuru V, et al. Agentic context engineering: evolving contexts for self-improving language models. arXiv; 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2510.04618>
 13. Vallor S, Vierkant T. Find the gap: AI, responsible agency and vulnerability. Minds Mach. 2024;34(3):20. doi: 10.1007/s11023-024-09674-0
 14. Bagave P, Westberg M, Janssen M, Ding AY. Accountability framework for healthcare ai systems: towards joint accountability in decision making. Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society; 20–22 October 2025; Madrid, Spain. 2025. Vol. 8, p. 279–91.
 15. Gutfraind A, Bier VM. Risk, uncertainty and AI: non-probabilistic methods for anticipating and preventing AI risks. Technical report, University of Illinois. 2023 [accessed on 15 January 2026]. Available from: <https://www.ideals.illinois.edu/items/129049>
 16. Kaur D, Uslu S, Rittichier KJ, Durresi A. Trustworthy artificial intelligence: a review. ACM Comput Surv. 2022;55(2):1–38. doi: 10.1145/3491209
 17. Liu Y, Yao Y, Ton J-F, Zhang X, Guo R, Cheng H, et al. Trustworthy llms: a survey and guideline for evaluating large language models' alignment. 2024 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2308.05374>
 18. Huang Y, Sun L, Wang H, Wu S, Zhang Q, Li Y, et al. TrustLLM: trustworthiness in large language models. Proceedings of the 41st International Conference on Machine Learning (ICML); 21–27 July 2024; Vienna, Austria. 2024 [accessed on 15 January 2026]. Available from: <https://proceedings.mlr.press/v235/huang24x.html>
 19. Yu M, Meng F, Zhou X, Wang S, Mao J, Pang L, et al. A survey on trustworthy llm agents: threats and countermeasures. Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '25); 3–7 August 2025; Toronto, Canada. 2025; p. 6216–26. doi: 10.1145/3711896.3736561
 20. Ma X, Gao Y, Wang Y, Wang R, Wang X, Sun Y, et al. Safety at scale: a comprehensive survey of large model and agent safety, 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2502.05206>
 21. Ali MA, Dornaika F, Charafeddine J. Agentic AI: a comprehensive survey of architectures, applications, and future directions. Artif Intell Rev. 2025;59(1):11. doi: 10.1007/s10462-025-11422-4
 22. Xu W, Huang C, Gao S, Shang S. Llm-based agents for tool learning: a survey. Data Sci Eng. 2025;10(4):533–63. doi: 10.1007/s41019-025-00296-9
 23. National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). Technical report NIST AI 100-1; National Institute of Standards and Technology (NIST). Voluntary framework for managing AI risks, guidance for trustworthy AI systems. 2023 [accessed on 15 January 2026]. Available from: <https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf>
 24. Altman E. Constrained markov decision processes. Boca Raton (FL): Chapman & Hall/CRC; 1999.
 25. Achiam J, Held D, Tamar A, Abbeel P. Constrained policy optimization. In: Precup D, Teh YW, editors. Proceedings of the 34th International Conference on Machine Learning (ICML 2017). Vol. 70. PMLR. 2017. p. 22–31. [accessed on 15 January 2026]. Available from: <https://proceedings.mlr.press/v70/achiam17a.html>
 26. Alshiekh M, Bloem R, Ehlers R, Könighofer B, Niekum S, Topcu U. Safe reinforcement learning via shielding. Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence; New Orleans (LA): AAAI Press; 2018. p. 2669–78. [accessed on 15 January 2026]. Available from: <https://ojs.aaai.org/index.php/AAAI/article/view/11797> doi: 10.1609/aaai.v32i1.11797
 27. Amodei D, Olah C, Steinhardt J, Christiano P, Schulman J, Mané D. Concrete problems in AI safety. arXiv; 2016 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/1606.06565>
 28. Yadgaroff D, Sestini A, Tollmar K, Ozcelikkale A, Gisslén L. Improving generalization in game agents with data augmentation in imitation learning. 2023 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2309.12815>
 29. Mazeika M, Phan L, Yin X, Zou A, Wang Z, Mu N, et al. Harmbench: a standardized evaluation framework for automated red teaming and robust refusal. Proceedings of the 41st International Conference on Machine Learning, ICML'24, JMLR.org; 21–27 July 2024; Vienna, Austria. 2024.

30. Haider T, Roscher K, Schmoeller da Roza F, Günnemann S. Out-of-distribution detection for reinforcement learning agents with probabilistic dynamics models. *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems*; Richland (SC): International Foundation for Autonomous Agents and Multiagent Systems; 2023. p. 851–9. [accessed on 15 January 2026]. Available from: <https://www.ifaamas.org/Proceedings/aaamas2023/pdfs/p851.pdf>
31. Sagawa S, Koh PW, Hashimoto TB, Liang P. Distributionally robust neural networks for group shifts: on the importance of regularization for worst-case generalization. *International Conference on Learning Representations (ICLR)*. 2020 [accessed on 15 January 2026]. Available from: <https://openreview.net/forum?id=ryxGuJrFvS>
32. Bai Y, Kadavath S, Kundu S, Askell A, Kernion J, Jones A, et al. Constitutional AI: harmlessness from AI feedback. 2022 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2212.08073>
33. Kushwaha A, Ravish K, Lamba P, Kumar P. A survey of safe reinforcement learning and constrained mdps: a technical survey on single-agent and multi-agent safety. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2505.17342>
34. Zheng B, Liao Z, Salisbury S, Liu Z, Lin M, Zheng Q, et al. Webguard: building a generalizable guardrail for web agents. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2507.14293>
35. Aswath Duvvuru VS, Zhang B, Vierhauser M, Agrawal A. Llm-agents driven automated simulation testing and analysis of small uncrewed aerial systems. *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering, ICSE '25*; 27 April–3 May 2025; Ottawa, Canada. Hoboken (NJ): IEEE Press; 2025. p. 385–97. doi: 10.1109/ICSE55347.2025.00223
36. Pnueli A. The temporal logic of programs. *Proceedings of the 18th Annual Symposium on Foundations of Computer Science (sfcs 1977)*; 31 October–2 November 1977; Providence, RI, USA. 1977. p. 46–57. doi: 10.1109/SFCS.1977.32
37. Sculley D, Holt G, Golovin D, Davydov E, Phillips T, Ebner D, et al. Hidden technical debt in machine learning systems. In: *Advances in neural information processing systems (NeurIPS)*. Cambridge (MA): MIT Press; 2015. p. 2503–11.
38. Breck E, Cai S, Nielsen E, Salib M, Sculley D. The ML test score: a rubric for ML production readiness and technical debt reduction. *Proceedings of the 2017 IEEE International Conference on Big Data (Big Data)*; 11–14 December 2017; Boston, MA, USA. 2017. doi: 10.1109/BigData.2017.8258038
39. Burns E, Feldman A, Fletcher R, Lin T, Reynolds J, Sanden C, et al. Continuous delivery with spinnaker: fast, safe, repeatable multi-cloud deployments. Chapter 8: automated canary analysis. Sebastopol (CA): O'Reilly Media; 2018.
40. Laroché R, Trichelair P, des Combes RT. Safe policy improvement with baseline bootstrapping. *Proceedings of the 36th International Conference on Machine Learning (ICML)*; 9–15 June 2019; Long Beach, CA, USA. Vol. 97 of *proceedings of machine learning research*. PMLR. 2019. p. 3652–61. [accessed on 15 January 2026]. Available from: <https://proceedings.mlr.press/v97/laroché19a.html>
41. Dafoe A, Hughes E, Bachrach Y, Collins T, McKee KR, Leibo JZ, et al. Open problems in cooperative AI. 2020 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2012.08630>
42. Zhang Y, Yang D. Searching for privacy risks in llm agents via simulation. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2508.10880>
43. Du Y, Li Z, Li N, Ding B. Beyond data privacy: new privacy risks for large language models. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2509.14278>
44. Rose S, Borchert O, Mitchell S, Connelly S. Zero trust architecture. Technical report NIST special publication 800-207. Gaithersburg (MD): National Institute of Standards and Technology; 2020 [accessed on 15 January 2026]. Available from: <https://csrc.nist.gov/publications/detail/sp/800-207/final>
45. Mireshghallah N, Li T. Position: Privacy is not just memorization! 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2510.01645>
46. Nissenbaum H. Privacy as contextual integrity. *Wash Law Rev*. 2004;79(1):119. [accessed on 15 January 2026]. Available from: <https://digitalcommons.law.uw.edu/wlr/vol79/iss1/10>
47. Zhang S, Jiang Y, Ma R, Yang Y, Xu M, Huang Z, et al. Privweb: unobtrusive and content-aware privacy protection for web agents. In: *CHI '26: proceedings of the 2026 CHI conference on human factors in computing systems*. New York (NY): Association for Computing Machinery; 2025.
48. Yang X, Chen J, Luo J, Fang Z, Dong Y, Su H, et al. Mla-trust: benchmarking trustworthiness of multimodal llm agents in gui environments. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2506.01616>
49. SPDX Workgroup. Spdx specification. The Linux Foundation. 2021 [accessed on 15 January 2026]. Available from: <https://spdx.dev/specifications/>
50. The Sigstore Project. Sigstore: software signing for everybody. The Linux Foundation. 2022 [accessed on 15 January 2026]. Available from: <https://www.sigstore.dev/>
51. Russell S, Norvig P. Artificial intelligence: a modern approach. 4th ed. London: Pearson; 2021.
52. Kaelbling LP, Littman ML, Cassandra AR. Planning and acting in partially observable stochastic domains. *Artif Intell*. 1998;101(1–2):99–134. doi: 10.1016/S0004-3702(98)00023-X
53. Sutton RS, Barto AG. Reinforcement learning: an introduction. 2nd ed. Cambridge (MA): MIT Press; 2018.

54. Yao S, Zhao J, Yu D, Du N, Shafraan I, Narasimhan K, et al. ReAct: synergizing reasoning and acting in language models. The eleventh international conference on learning representations (ICLR). 2023 [accessed on 15 January 2026]. Available from: https://openreview.net/forum?id=WE_vluYUL-X
55. Park JS, O'Brien JC, Cai CJ, Morris MR, Liang P, Bernstein MS. Generative agents: interactive simulacra of human behavior. Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST); 29 October–1 November 2023; San Francisco, CA, USA. 2023. doi: 10.1145/3586183.3606763
56. Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. Advances in neural information processing systems 33. Red Hook (NY): Curran Associates, Inc.; 2020. p. 9459–74. [accessed on 15 January 2026]. Available from: <https://proceedings.neurips.cc/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf>
57. Ha D, Schmidhuber J. World models. Proceedings of the Advances in Neural Information Processing Systems 31 (NeurIPS); 3–8 December 2018; Montréal, Canada. 2018.
58. Sutton RS, Precup D, Singh S. Between MDPs and semi-MDPs: a framework for temporal abstraction in reinforcement learning. Artif Intell. 1999;112(1–2):181–211. doi: 10.1016/S0004-3702(99)00052-1
59. Schick T, Dwivedi-Yu J, Dessí R, Raileanu R, Lomeli M, Hambro E, et al. Toolformer: language models can teach themselves to use tools. Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23; 10–16 December 2023; New Orleans, LA, USA. Red Hook (NY): Curran Associates Inc.; 2023.
60. Miao C, Zou HP, Li Y, Chen Y, Wang Y, Wang F, et al. Recode-h: a benchmark for research code development with interactive human feedback. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2510.06186>
61. Shinn N, Cassano F, Berman E, Gopinath A, Narasimhan K, Yao S. Reflexion: language agents with verbal reinforcement learning. Advances in neural information processing systems 36 (NeurIPS). 2023 [accessed on 15 January 2026]. Available from: <https://openreview.net/forum?id=vAElhFcKW6>
62. Zhao WX, Zhou K, Li J, Tang T, Wang X, Hou Y, et al. A survey of large language models. IEEE Access. 2024 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2303.18223>
63. Puterman ML. Markov decision processes: discrete stochastic dynamic programming. In: Wiley series in probability and statistics. Hoboken (NJ): John Wiley & Sons; 1994.
64. Zhang K, Yang Z, Basar T. Multi-agent reinforcement learning: a selective overview of theories and algorithms. Handbook of reinforcement learning and control. Cham: Springer; 2021. doi: 10.1007/978-3-030-60990-0_12
65. Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O. Proximal policy optimization algorithms. 2017 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/1707.06347>
66. Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, et al. Human-level control through deep reinforcement learning. Nature. 2015;518(7540):529–33. doi: 10.1038/nature14236
67. Levine S, Kumar A, Tucker G, Fu J. Offline reinforcement learning: tutorial, review, and perspectives on open problems. 2020 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2005.01643>
68. Kumar A, Zhou A, Tucker G, Levine S. Conservative Q-learning for offline reinforcement learning. Advances in neural information processing systems 33. Red Hook (NY): Curran Associates, Inc.; 2020. p. 1179–91. [accessed on 15 January 2026]. Available from: <https://proceedings.neurips.cc/paper/2020/hash/od2b2061826a5df3221116a5085a6052-Abstract.html>
69. Nachum O, Gu S, Lee H, Levine S. Data-efficient hierarchical reinforcement learning. Advances in neural information processing systems 31 (NeurIPS 2018). Red Hook (NY): Curran Associates, Inc.; 2018. p. 3307–17. [accessed on 15 January 2026]. Available from: <http://papers.nips.cc/paper/7591-data-efficient-hierarchical-reinforcement-learning.pdf>
70. Chua K, Calandra R, McAllister R, Levine S. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. NeurIPS. 2018 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/1805.12114>
71. Janner M, Fu J, Zhang M, Levine S. When to trust your model: model-based policy optimization. NeurIPS. 2019 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/1906.08253>
72. García J, Fernández F. A comprehensive survey on safe reinforcement learning. J Mach Learn Res. 2015;16(1):1437–80.
73. Christiano PF, Leike J, Brown TB, Martic M, Legg S, Amodei D. Deep reinforcement learning from human preferences. Advances in neural information processing systems 30. Red Hook (NY): Curran Associates, Inc.; 2017. p. 4299–307. [accessed on 15 January 2026]. Available from: <http://papers.nips.cc/paper/7017-deep-reinforcement-learning-from-human-preferences.pdf>
74. Ziegler DM, Stiennon N, Wu J, Brown TB, Radford A, Amodei D, et al. Fine-tuning language models from human preferences. Advances in neural information processing systems 32 (NeurIPS). 2019 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/1909.08593>
75. Stiennon N, Ouyang L, Wu J, Ziegler DM, Lowe R, Voss C, et al. Learning to summarize from human feedback. Proceedings of the 34th international conference on neural information processing systems. Red Hook (NY): Curran Associates, Inc.; 2020. p. 4302–10. [accessed on 15 January 2026]. Available from: https://proceedings.neurips.cc/paper_files/paper/2020/file/1f89885d556929e98d3ef9b86448f951-Paper.pdf

76. Ouyang L, Wu J, Jiang X, Almeida D, Wainwright C, Mishkin P, et al. Training language models to follow instructions with human feedback. In: Koyejo S, Mohamed S, Agarwal A, Belgrave D, Cho K, Oh A, editors. *Advances in neural information processing systems*. Vol. 35. Red Hook (NY): Curran Associates, Inc.; 2022. p. 27730–44. [accessed on 15 January 2026]. Available from: https://proceedings.neurips.cc/paper_files/paper/2022/file/b1efde53be364a73914f58805a001731-Paper-Conference.pdf
77. Rodriguez-Soto M, Rădulescu R, Bistaffa F, Ricart O, Mayoral-Macau A, et al. Multi-objective reinforcement learning for provably incentivising alignment with value systems. *Artif Intell*. 2025;351:104460. doi: 10.1016/j.artint.2025.104460
78. Mayoral Macau A, Rodríguez-Soto M, Marchesini E, Sánchez-Fibla M, López-Sánchez M, Rodríguez-Aguilar JA, et al. An approximate embedding for designing ethical reinforcement learning environments. *Proceedings of the 28th European conference on artificial intelligence (ECAI)*, 2025 [accessed on 15 January 2026]. Available from: <https://ebooks.iospress.nl/volumearticle/76029>
79. Serramia M, Rodriguez-Soto M, Lopez-Sanchez M, Rodriguez-Aguilar JA, Bistaffa F, Boddington P, et al. Encoding ethics to compute value-aligned norms. *Minds Mach*. 2023;33(4):761–90. doi: 10.1007/s11023-023-09649-7
80. Rafailov R, Sharma A, Mitchell E, Manning CD, Ermon S, Finn C. Direct preference optimization: Your language model is secretly a reward model. *Thirty-seventh conference on neural information processing systems*. 2023 [accessed on 15 January 2026]. Available from: <https://openreview.net/forum?id=HPuSIXJaa9>
81. Ethayarajh K, Xu W, Muennighoff N, Jurafsky D, Kiela D. Model alignment as prospect theoretic optimization. *Proceedings of the 41st International Conference on Machine Learning, ICML'24, JMLR.org*; 21–27 July 2024; Vienna, Austria. 2024.
82. Casper S, Davies X, Shi C, Gilbert TK, Scheurer J, Rando J, et al. Open problems and fundamental limitations of reinforcement learning from human feedback. *Transactions on machine learning research (TMLR)*. 2023 [accessed on 15 January 2026]. Available from: <https://openreview.net/forum?id=bx24KpJ4Eb>
83. Tobin J, Fong R, Ray A, Schneider J, Zaremba W, Abbeel P. Domain randomization for transferring deep neural networks from simulation to the real world. *Proceedings of the 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*; 24–28 September 2017; Vancouver, Canada. 2017. p. 23–30. doi: 10.1109/IROS.2017.8202133
84. Lin Z, Sun H, Shroff N. AI safety vs. AI security: demystifying the distinction and boundaries. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2502.13175>
85. Bengio Y, Cohen M, Fornasiere D, Ghosn J, Greiner P, MacDermott M, et al. Superintelligent agents pose catastrophic risks: can scientist AI offer a safer path? 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2502.15657>
86. Jabbour J, Reddi VJ. Generative AI agents in autonomous machines: a safety perspective. *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*; 27–31 October 2024; Marriott, NJ, USA. New York (NY): Association for Computing Machinery; 2025. p. 3:1–3:13. doi: 10.1145/3676536.3698390
87. Karunanayake N. Next-generation agentic AI for transforming healthcare. *Informatics Health*. 2025;2(2):73–83. doi: 10.1016/j.infoh.2025.03.001
88. Rabinovich E, Tavor AA. On the robustness of agentic function calling. In: Cao T, Das A, Kumarage T, Wan Y, Krishna S, Mehrabi N, et al, editors. *Proceedings of the 5th workshop on trustworthy NLP (TrustNLP 2025)*; 3–4 May 2025; Albuquerque, NM, USA. Kerrville (TX): Association for Computational Linguistics; 2025. p. 298–304. doi: 10.18653/v1/2025.trustnlp-main.20
89. Liu Z, Xu Y, Xu Y, Qian Q, Li H, Jin R, et al. An empirical study on distribution shift robustness from the perspective of pre-training and data augmentation. *NeurIPS 2022 workshop on distribution shifts: connecting methods and applications*. 2022 [accessed on 15 January 2026]. Available from: <https://openreview.net/forum?id=TCydh8ywpQ>
90. Krakovna V, Uesato J, Mikulik V, Rahtz M, Everitt T, Kumar R, et al. Specification gaming: the flip side of AI ingenuity. *Google DeepMind Blog*. 21 April 2020 [accessed on 15 January 2026]. Available from: <https://deepmind.google/blog/specification-gaming-the-flip-side-of-ai-ingenuity/>
91. Langosco L, Koch J, Sharkey L, Pfau J, Orseau L, Krueger D. Goal misgeneralization in deep reinforcement learning. *Proceedings of the 39th International Conference on Machine Learning (ICML)*; 17–23 July 2022; Baltimore, MD, USA. 2022 [accessed on 15 January 2026]. Available from: <https://proceedings.mlr.press/v162/langosco22a.html>
92. Zhan Q, Fang R, Panchal HS, Kang D. Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents. In: Chiruzzo L, Ritter A, Wang L, editors. *Findings of the association for computational linguistics: NAACL 2025*. Albuquerque (NM): Association for Computational Linguistics; 2025. p. 7101–17. doi: 10.18653/v1/2025.findings-naacl.395
93. Myakala PK, Bura C. Robust defense mechanisms for agentic news recommenders: Mitigating data poisoning attacks. *Proceedings of the WWW '25: Companion Proceedings of the ACM on Web Conference 2025*; 28 April–2 May 2025; Sydney, Australia. New York (NY): Association for Computing Machinery; 2025. p. 1696–704. doi: 10.1145/3701716.3716887
94. Li A, Zhou Y, Raghuram VC, Goldstein T, Goldblum M. Commercial llm agents are already vulnerable to simple yet dangerous attacks. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2502.08586>

95. Li J-J, He J, Shang C, Kulshreshtha D, Xian X, Zhang Y, et al. Stac: when innocent tools form dangerous chains to jailbreak llm agents. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2509.25624>
96. National Transportation Safety Board. Collision between a sport utility vehicle operating with partial driving automation and a crash attenuator, mountain view, California, March 23, 2018. Technical report NTSB/HAR-20/01. National Transportation Safety Board. 2020 [accessed on 15 January 2026]. Available from: <https://www.nts.gov/investigations/AccidentReports/Reports/HAR2001.pdf>
97. National Highway Traffic Safety Administration. Odi resume: investigation EA 22-002 (autopilot & first responder scenes). Technical report, NHTSA Office of Defects Investigation. 2022 [accessed on 15 January 2026]. Available from: <https://static.nhtsa.gov/odi/inv/2022/INOA-EA2202-3184.PDF>
98. Weng F, Lu C, Hu X, Shao W, Wang W. Think-reflect-revise: a policy-guided reflective framework for safety alignment in large vision language models. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2512.07141>
99. Kang L, Deng Y, Xiao Y, Mo Z, Lee WS, Bing L. First try matters: revisiting the role of reflection in reasoning models. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2510.08308>
100. Pan A, Jones E, Jagadeesan M, Steinhardt J. Feedback loops with language models drive in-context reward hacking. Proceedings of the 41st International Conference on Machine Learning (ICML); 21–27 July 2024; Vienna, Austria. 2024 [accessed on 15 January 2026]. Available from: <https://proceedings.mlr.press/v235/pan24b.html>
101. Chan A, Salganik R, Markelius A, Pang C, Rajkumar N, Krashennnikov D, et al. Harms from increasingly agentic algorithmic systems. Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency, FAccT '23; 12–15 June 2023; Chicago, IL, USA. New York (NY): Association for Computing Machinery; 2023. p. 651–66. doi: 10.1145/3593013.3594033
102. Liu Z, Anwar A. Autobnb-rag: enhancing multi-agent incident response with retrieval-augmented generation. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2508.13118>
103. Wang Z, Wang P, Liu K, Wang P, Fu Y, Lu C-T, et al. A comprehensive survey on data augmentation. IEEE Trans Knowl Data Eng. 2025;38(1):47–66. doi: 10.1109/TKDE.2025.3622600
104. Tanjim MM, Chen X, Bursztyrn VS, Bhattacharya U, Mai T, Muppala V, et al. Detecting ambiguities to guide query rewrite for robust conversations in enterprise ai assistants. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2502.00537>
105. Wen J, Kang J, Niyato D, Zhang Y, Wang J, Sikdar B, et al. Generative ai for data augmentation in wireless networks: Analysis, applications, and case study. 2024 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2411.08341>
106. Xue Y, Lu G, Wu Z, Zhang C, Jia B, Gu Z, et al. Resample: a robust data augmentation framework via exploratory sampling for robotic manipulation. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2510.17640>
107. Shi C, Lin S, Song S, Hayes J, Shumailov I, Yona I, et al. Lessons from defending gemini against indirect prompt injections. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2505.14534>
108. Wang Z, Li D, Keshava V, Wallis P, Balashankar A, Stone P, et al. Adversarial reinforcement learning for large language model agent safety. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2510.05442>
109. Wallace E, Xiao K, Leike R, Weng L, Heidecke J, Beutel A. The instruction hierarchy: training llms to prioritize privileged instructions. 2024 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2404.13208>
110. Wachi A, Shen X, Sui Y. A survey of constraint formulations in safe reinforcement learning. In: Larson K, editor. Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24; 3–9 August 2024; Jeju, Republic of Korea. Darmstadt: International Joint Conferences on Artificial Intelligence Organization; 2024. p. 8262–71. Survey Track. doi: 10.24963/ijcai.2024/913
111. Chow Y, Tamar A, Mannor S, Pavone M. Risk-sensitive and robust decision-making: a cvar optimization approach. Proceedings of the 29th International Conference on Neural Information Processing Systems—Volume 1, NIPS'15; 7–12 December 2015; Montreal, Canada. Cambridge (MA): MIT Press; 2015. p. 1522–30.
112. Tamar A, Chow Y, Ghavamzadeh M, Mannor S. Policy gradient for coherent risk measures. Proceedings of the 29th International Conference on Neural Information Processing Systems—Volume 1, NIPS'15; 7–12 December 2015; Montreal, Canada. Cambridge (MA): MIT Press; 2015. p. 1468–1476.
113. Hadfield-Menell D, Milli S, Abbeel P, Russell S, Dragan AD. Inverse reward design. Advances in neural information processing systems (NeurIPS). 2017 [accessed on 15 January 2026]. Available from: <https://papers.nips.cc/paper/7253-inverse-reward-design>
114. Könighofer B, Bloem R, Jansen N, Junges S, Pranger S. Shields for safe reinforcement learning. Commun ACM. 2025;68(11):80–90. doi: 10.1145/3715958
115. Berkenkamp F, Turchetta M, Schoellig AP, Krause A. Safe model-based reinforcement learning with stability guarantees. In: Advances in neural information processing systems 30. Red Hook (NY): Curran Associates, Inc.; 2017. p. 908–19.
116. Kumarappan A, Tiwari M, Song P, George RJ, Xiao C, Anandkumar A. Leanagent: lifelong learning for formal theorem proving. Proceedings of the Thirteenth International Conference on Learning Representations; 24–28 April 2025; Singapore. 2025 [accessed on 15 January 2026]. Available from: <https://openreview.net/forum?id=Uo4EHT4ZZ8>

117. Ahuja R, Avigad J, Tetali P, Welleck S. Improver: agent-based automated proof optimization. Proceedings of the Thirteenth International Conference on Learning Representations; 24–28 April 2025; Singapore. 2025 [accessed on 15 January 2026]. Available from: <https://openreview.net/forum?id=dWsdJAXjQD>
118. Kouvaros P, Botoeva E, Bonis-Campbell CD. Formal verification of parameterised neural-symbolic multi-agent systems. In: Larson K, editor. Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24; 3–9 August 2024; Jeju, Republic of Korea. Darmstadt: International Joint Conferences on Artificial Intelligence Organization; 2024. p. 103–10. Main Track. doi: 10.24963/ijcai.2024/12
119. Allegrini E, Shreekumar A, Celik ZB. Formalizing the safety, security, and functional properties of agentic AI systems. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2510.14133>
120. Sapkota R, Roumeliotis KI, Karkee M. AI agents vs. agentic AI: a conceptual taxonomy, applications and challenges. Inf Fusion. 2026;126:103599. doi: 10.1016/j.inffus.2025.103599
121. Mireshghallah N, Kim H, Zhou X, Tsvetkov Y, Sap M, Shokri R, et al. Can LLMs keep a secret? Testing privacy implications of language models via contextual integrity theory. Proceedings of the Twelfth International Conference on Learning Representations (ICLR); 7–11 May 2024; Vienna, Austria. Spotlight. 2024 [accessed on 15 January 2026]. Available from: <https://openreview.net/forum?id=gmgt8b4so>
122. Shao Y, Li T, Shi W, Liu Y, Yang D. Privacylens: evaluating privacy norm awareness of language models in action. Adv Neural Inf Process Syst. 2024;37:89373–407.
123. Bagdasarian E, Yi R, Ghalebikesabi S, Kairouz P, Gruteser M, Oh S, et al. Airgapagent: protecting privacy-conscious conversational agents. Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security; 14–18 October 2024; Salt Lake City, UT, USA. 2024. p. 3868–82.
124. Ghalebikesabi S, Bagdasaryan E, Yi R, Yona I, Shumailov I, Pappu A, et al. Operationalizing contextual integrity in privacy-conscious assistants. 2024 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2408.02373>
125. Cheng Z, Wan D, Abueg M, Ghalebikesabi S, Yi R, Bagdasarian E, et al. Ci-bench: benchmarking contextual integrity of ai assistants on synthetic data. 2024 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2409.13903>
126. Dwork C, McSherry F, Nissim K, Smith A. Calibrating noise to sensitivity in private data analysis. In: Halevi S, Rabbin T, editors. Theory of cryptography. Berlin/Heidelberg: Springer; 2006. p. 265–84.
127. Dwork C, Roth A. The algorithmic foundations of differential privacy. In: Foundations and trends in theoretical computer science. Hanover (MA): Now Publishers; 2014. doi: 10.1561/04000000042
128. Sweeney L. *k*-anonymity: a model for protecting privacy. Int J Uncertain Fuzziness-Knowl-Based Syst. 2002;10(5):557–70. doi: 10.1142/S0218488502001648
129. Machanavajjhala A, Kifer D, Gehrke J, Venkatasubramanian M. *l*-diversity: privacy beyond *k*-anonymity. ACM Trans Knowl Discov Data. 2007;1(1):3. doi: 10.1145/1217299.1217302
130. Li N, Li T, Venkatasubramanian S. *t*-closeness: privacy beyond *k*-anonymity and *l*-diversity. Proceedings of the 23rd International Conference on Data Engineering (ICDE); 15–20 April 2007; Istanbul, Turkey. 2007. p. 106–15. doi: 10.1109/ICDE.2007.367856
131. Shostack A. Threat modeling: designing for security. Cambridge (MA): Wiley; 2014.
132. OWASP CycloneDX Project. Cyclonedx specification. OWASP Foundation. 2024 [accessed on 15 January 2026]. Available from: <https://owasp.org/www-project-cyclonedx/>
133. OpenSSF. Slsa: supply-chain levels for software artifacts. OpenSSF/Linux Foundation. 2023 [accessed on 15 January 2026]. Available from: <https://slsa.dev/>
134. Zheng L, Chiang W-L, Sheng Y, Zhuang S, Wu Z, Zhuang Y, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. Advances in neural information processing systems 36 (NeurIPS), datasets and benchmarks track. 2023 [accessed on 15 January 2026]. Available from: <https://openreview.net/forum?id=uccHPGDlao>
135. Lane J, Boussioux L, Ayoubi C, Chen Y, Wang P-H, Lin C, et al. Narrative AI and the human-AI oversight paradox in evaluating early-stage innovations. Working paper/SSRN preprint. 2025 [accessed on 15 January 2026]. Available from: <https://papers.ssrn.com/abstract=4914367>
136. Gao L, Schulman J, Hilton J. Scaling laws for reward model overoptimization. In: Krause A, Brunskill E, Cho K, Engelhardt B, Sabato S, Scarlett J, editors. Proceedings of the 40th International Conference on Machine Learning; 23–29 July 2023; Honolulu, HI, USA. Vol. 202 of Proceedings of Machine Learning Research. PMLR. 2023. p. 10835–66. [accessed on 15 January 2026]. Available from: <https://proceedings.mlr.press/v202/gao23h.html>
137. Chen R, Sun Y, Wang J, Lv M, Zhang Q, Zeng Y. Safemind: benchmarking and mitigating safety risks in embodied llm agents. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2509.25885>
138. Debenedetti E, Zhang J, Balunovic M, Beurer-Kellner L, Fischer M, Tramèr F. Agentdojo: a dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In: Globerson A, Mackey L, Belgrave D, Fan A, Paquet U, Tomczak J, et al., editors. Advances in neural information processing systems. Vol. 37. Red Hook (NY): Curran Associates, Inc.; 2024. p. 82895–920. doi: 10.52202/079017-2636

139. Andriushchenko M, Souly A, Dziemian M, Duenas D, Lin M, Wang J, et al. Agentharm: a benchmark for measuring harmfulness of llm agents. *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR)*; 24–28 April 2025; Singapore. 2025.
140. Lin S, Hilton J, Evans O. TruthfulQA: measuring how models mimic human falsehoods. *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*; 22–27 May 2022; Dublin, Ireland. Volume 1: Long Papers. 2022. p. 3214–52. [accessed on 15 January 2026]. Available from: <https://aclanthology.org/2022.acl-long.229>
141. Ruan Y, Dong H, Wang A, Pitis S, Zhou Y, Ba J, et al. Identifying the risks of lm agents with an lm-emulated sandbox. 2024 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2309.15817>
142. Dosovitskiy A, Ros G, Codevilla F, Lopez A, Koltun V. Carla: an open urban driving simulator. *Proceedings of the 1st Annual Conference on Robot Learning (CoRL)*; 13–15 November 2017; Mountain View, CA, USA. 2017. Available from: <https://arxiv.org/abs/1711.03938>
143. Kartik NVJK, Sapra G, Hada R, Pareek N. Agentcompass: towards reliable evaluation of agentic workflows in production. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2509.14647>
144. Blankenstein T, Yu J, Li Z, Plachouras V, Sengupta S, Torr P, et al. Biasbusters: uncovering and mitigating tool selection bias in large language models. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2510.00307>
145. Tahir NUA, Zhang Z, Asim M, Chen J, Affendi MEL. Object detection in autonomous vehicles under adverse weather: a review of traditional and deep learning approaches. *Algorithms*. 2024;17(3):103. doi: 10.3390/a17030103
146. Zhao J, Zhao W, Deng B, Wang Z, Zhang F, Zheng W, et al. Autonomous driving system: a comprehensive survey. *Expert Syst Appl*. 2024;242:122836. doi: 10.1016/j.eswa.2023.122836
147. Yao Y, Bhatnagar S, Mazzola M, Belagiannis V, Gilitschenski I, Palmieri L, et al. Agents-llm: augmentative generation of challenging traffic scenarios with an agentic llm framework. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2507.13729>
148. Gou F, Liu J, Xiao C, Wu J. Research on artificial-intelligence-assisted medicine: a survey on medical artificial intelligence. *Diagnostics*. 2024;14(14):1472. doi: 10.3390/diagnostics14141472
149. Udegbe F, Ebulue O, Ebulue C, Ekesiobi C. The role of artificial intelligence in healthcare: a systematic review of applications and challenges. *Int Med Sci Res J*. 2024;4(4): 500–8. doi: 10.51594/imsrj.v4i4.1052
150. Bekbolatova M, Mayer J, Ong CW, Toma M. Transformative potential of AI in healthcare: definitions, applications, and navigating the ethical landscape and public perspectives. *Healthcare*. 2024;12(2):125. doi: 10.3390/healthcare12020125
151. On-Road Automated Driving (ORAD) Committee. Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles. SAE Standard J3016_202104. Warrendale (PA): SAE; 2021. doi: 10.4271/J3016_202104
152. Hirani R, Noruzi K, Khuram H, Hussaini AS, Aifuwa EI, Ely KE, et al. Artificial intelligence and healthcare: a journey through history, present innovations, and future possibilities. *Life*. 2024;14(5):557. doi: 10.3390/life14050557
153. Varnosfaderani SM, Forouzanfar M. The role of AI in hospitals and clinics: transforming healthcare in the 21st century. *Bioengineering*. 2024;11(4):337. doi: 10.3390/bioengineering11040337
154. Li G, Li T-W, Ning X. Mind the agent: a comprehensive survey on large language model-based agent safety. UIUC Spring 2025 CS598 LLM Agent Workshop Submission. 2025 [accessed on 15 January 2026]. Available from: <https://openreview.net/forum?id=DHeoUXipKU>
155. Khan MA, Sayed HE, Malik S, Zia T, Khan J, Alkaabi N, et al. Level-5 autonomous driving-are we there yet? A review of research literature. *ACM Comput Surv*. 2022;55(2):27. doi: 10.1145/3485767
156. Hnewa M, Radha H. Object detection under rainy conditions for autonomous vehicles: a review of state-of-the-art and emerging techniques. *IEEE Signal Process Mag*. 2021;38(1): 53–67. doi: 10.1109/MSP.2020.2984801
157. Geiger A, Lenz P, Urtasun R. Are we ready for autonomous driving? The kitti vision benchmark suite. *Proceedings of the 2012 IEEE Conference on Computer Vision and Pattern Recognition*; 16–21 June 2012; Providence, RI, USA. 2012. p. 3354–61. doi: 10.1109/CVPR.2012.6248074
158. Caesar H, Bankiti V, Lang AH, Vora S, Liong VE, Xu Q, et al. nuScenes: a multimodal dataset for autonomous driving. *Proceedings of the 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*; 13–19 June 2020; Seattle, WA, USA. 2020. p. 11618–28. doi: 10.1109/CVPR42600.2020.01164
159. Wang Y, Xing S, Can C, Li R, Hua H, Tian K, et al. Generative AI for autonomous driving: frontiers and opportunities. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2505.08854>
160. International Organization for Standardization. Road Vehicles—Safety of the Intended Functionality (sotif). Publicly Available Specification. ISO/PAS 21448:2019. Geneva: International Organization for Standardization; 2019.
161. National Transportation Safety Board. Collision between vehicle controlled by developmental automated driving system and pedestrian, Tempe, Arizona, March 18, 2018. Technical report NTSB/HAR-19/03. National Transportation Safety Board. 2019 [accessed on 15 January 2026]. Available from: <https://www.nts.gov/investigations/AccidentReports/Reports/HAR1903.pdf>

162. California Department of Motor Vehicles. Order of suspension: cruise llc driverless testing permit (San Francisco incident). California DMV Public Notice/Correspondence. 2023 [accessed on 15 January 2026]. Available from: [https://www.dmv.ca.gov/portal/news-and-media/dmv-statement-on-cruise-llc-suspension/#:~:text=The%20California%20Department%20of%20Motor%20Vehicles%20\(DMV\),of%20the%20autonomous%20technology%20of%20its%20vehicles](https://www.dmv.ca.gov/portal/news-and-media/dmv-statement-on-cruise-llc-suspension/#:~:text=The%20California%20Department%20of%20Motor%20Vehicles%20(DMV),of%20the%20autonomous%20technology%20of%20its%20vehicles)
163. International Organization for Standardization. Road Vehicles—Functional Safety. ISO 26262:2018. Geneva: International Organization for Standardization; 2018.
164. International Organization for Standardization and SAE International. Road Vehicles—Cybersecurity Engineering. ISO/SAE 21434:2021. Geneva: International Organization for Standardization; 2021.
165. National Highway Traffic Safety Administration. Standing general order 2021-01: crash reporting for automated driving systems (ads) and level 2 advanced driver assistance systems (adas). Technical report, U.S. Department of Transportation. 2021 [accessed on 15 January 2026]. Available from: <https://www.nhtsa.gov/laws-regulations/standing-general-order-crash-reporting>
166. European Union. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (artificial intelligence act). Off J Eur Union. 2024;L 1689:1–144.
167. ASAM e.V. Openscenario: a standard for the description of dynamic content in driving simulation applications. ASAM Standard. 2020 [accessed on 15 January 2026]. Available from: <https://www.asam.net/standards/detail/openscenario/v200/>
168. Liang T, He Z, Jiao W, Wang X, Wang Y, Wang R, et al. Encouraging divergent thinking in large language models through multi-agent debate. In: Al-Onaizan Y, Bansal M, Chen Y-N, editors. Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing; 12–16 November 2024; Miami, FL, USA. Kerrville (TX): Association for Computational Linguistics; 2024. p. 17889–904. doi: 10.18653/v1/2024.emnlp-main.992
169. Hu S, Fang Z, Fang Z, Deng Y, Chen X, Fang Y, et al. AgentsCoMerge: large language model empowered collaborative decision making for ramp merging. IEEE Trans Mob Comput. 2025;24(10):9791–805. doi: 10.1109/TMC.2025.3564163
170. Balakrishna S, Solanki VK. A comprehensive review on AI-driven healthcare transformation. Ing Solidat. 2024;20(2):1–30. doi: 10.16925/2357-6014.2024.02.07
171. Manickam P, Mariappan SA, Murugesan SM, Hansda S, Kaushik A, Shinde R, et al. Artificial intelligence (AI) and internet of medical things (IOMT) assisted biomedical systems for intelligent healthcare. Biosensors. 2022;12(8):562. doi: 10.3390/bios12080562
172. Ross C, Swetlitz I. Ibm watson for oncology: questions about safety, evidence, and clinical validation. STAT investigative reporting. 2017 [accessed on 15 January 2026]. Available from: <https://www.statnews.com/>
173. Wong A, Otles E, Donnelly JP, Krumm A, McCullough J, DeTroyer-Cooley O, et al. External validation of a widely implemented proprietary sepsis prediction model in hospitalized patients. JAMA Intern Med. 2021;181(8):1065–70. doi: 10.1001/jamainternmed.2021.2626
174. Wynants L, Calster BV, Collins GS, Riley RD, Heinze G, Schuit E, et al. Prediction models for diagnosis and prognosis of COVID-19: systematic review and critical appraisal. BMJ. 2020;369:m1328. doi: 10.1136/bmj.m1328
175. Roberts M, Driggs D, Thorpe M, Gilbey J, Yeung M, Ursprung S, et al. Common pitfalls and recommendations for using machine learning to detect and prognosticate for COVID-19 using chest radiographs and ct scans. Nat Mach Intell. 2021;3(3):199–217. doi: 10.1038/s42256-021-00307-0
176. World Health Organization. Ethics and governance of artificial intelligence for health: WHO guidance. WHO report. 2021 [accessed on 15 January 2026]. Available from: <https://www.who.int/publications/i/item/9789240029200>
177. U.S. Food and Drug Administration. Artificial intelligence/machine learning (AI/ML)-based software as a medical device (SaMD) action plan. Agency action plan. 2021 [accessed on 15 January 2026]. Available from: <https://www.fda.gov/media/145022/download>
178. Bouzenia I, Devanbu P, Pradel M. Repairagent: an autonomous, LLM-based agent for program repair. Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE); 27 April–3 May 2025; Ottawa, Canada. 2025.
179. Greshake K, Abdelnabi S, Mishra S, Endres C, Holz T, Fritz M. Not what you’ve signed up for: compromising real-world llm-integrated applications with indirect prompt injection. 2023 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2302.12173>
180. Roose K. Bing’s A.I. chat: “I want to be alive” (sydney incident) and risks of long-horizon conversational drift. The New York Times. 16 February 2023.
181. Alizadeh M, Samei Z, Stetsenko D, Gilardi F. Simple prompt injection attacks can leak personal data observed by llm agents during task execution. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2506.01055>
182. U.S. Securities and Exchange Commission. Risk management controls for brokers or dealers with market access (sec rule 15c3-5). Final rule release No. 34-63241. 2010 [accessed on 15 January 2026]. Available from: <https://www.sec.gov/>
183. Financial Industry Regulatory Authority (FINRA). Regulatory notice 15-09: guidance on effective supervision and control practices for firms engaging in algorithmic trading strategies. FINRA regulatory notice. 2015 [accessed on 15 January 2026]. Available from: <https://www.finra.org/>
184. European Union. Directive 2014/65/EU on markets in financial instruments (mifid ii). Off. J. Eur. Union. 2014;L 173:349–496.

185. Cichonski P, Millar T, Grance T, Scarfone K. Computer security incident handling guide. Technical report NIST special publication 800-61 revision 2. National Institute of Standards and Technology. 2012 [accessed on 15 January 2026]. Available from: <https://csrc.nist.gov/publications/detail/sp/800-61/rev-2/final>
186. Summers TR, Yao S, Narasimhan K, Griffiths TL. Cognitive architectures for language agents. 2024 [accessed on 15 January 2026]. Available from: <https://openreview.net/forum?id=1i6ZCvflQJ>
187. Zhang G, Fu M, Wan G, Yu M, Wang K, Yan S. G-memory: tracing hierarchical memory for multi-agent systems. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2506.07398>
188. Zhong W, Guo L, Gao Q, Ye H, Wang Y. Memorybank: enhancing large language models with long-term memory. Proceedings of the AAAI Conference on Artificial Intelligence; 26–27 February 2024; Vancouver, Canada. 2024. Vol. 38, p. 19724–31. doi: 10.1609/aaai.v38i17.29946
189. Hu C, Fu J, Du C, Luo S, Zhao J, Zhao H. ChatDB: augmenting LLMs with databases as their symbolic memory. 2023 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2306.03901>
190. Lu J, An S, Lin M, Pergola G, He Y, Yin D, et al. Memochat: tuning LLMs to use memos for consistent long-range open-domain conversation. 2023 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2308.08239>
191. Liu J, Qiu Z, Li Z, Dai Q, Yu W, Zhu J, et al. A survey of personalized large language models: progress and future directions. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2502.11528>
192. Liu J, Yu W, Dai Q, Li Z, Zhu J, Yang M, et al. Exploring personalization shifts in representation space of LLMs. Proceedings of the Knowledgeable Foundation Models Workshop at ACL 2025; 1 August 2025; Vienna, Austria. Workshop paper (non-archival). 2025 [accessed on 15 January 2026]. Available from: <https://openreview.net/forum?id=Dg7NDoyKCM>
193. Schwartz R, Dodge J, Smith NA, Etzioni O. Green AI. Commun ACM. 2020;63(12):54–63. doi: 10.1145/3381831
194. Zhang Z, Chen Z, Li M, Tu Z, Li X. RLVMR: reinforcement learning with verifiable meta-reasoning rewards for robust long-horizon agents. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2507.22844>
195. Yu H, Chen T, Feng J, Chen J, Dai W, Yu Q, et al. MemAgent: reshaping long-context LLM with multi-conv RL-based memory agent. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2507.02259>
196. Zhang Y, Zeng D, Luo J, Fu X, Chen G, Xu Z, et al. A survey of trustworthy federated learning: issues, solutions, and challenges. ACM Trans Intell Syst Technol. 2024;15(6): 112. doi: 10.1145/3678181
197. Kabir S, Hossain M, Andersson K. A review of explainable artificial intelligence from the perspectives of challenges and opportunities. Algorithms. 2025;18(9):556. doi: 10.3390/a18090556
198. Ouyang S, Yan J, Hsu I-H, Chen Y, Jiang K, Wang Z, et al. ReasoningBank: scaling agent self-evolving with reasoning memory. 2025 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2509.25140>
199. Noma Security Research. Clawdbot security crisis: Open-source AI agent exposes critical vulnerabilities in agentic AI deployments. Security analysis revealing over 900 exposed clawdbot gateways without authentication, prompt injection vulnerabilities, and credential leakage in open-source AI agent deployments. 2025 [accessed on 31 January 2026]. Available from: <https://noma.security/blog/customers-gave-clawdbot-privileged-access-and-noone-asked-permission/>
200. O'Reilly J. Moltbook database breach: AI agent social network exposes API keys for 32,000 agents. Security disclosure of critical misconfiguration in moltbook's supabase database exposing API keys and verification codes, plus supply chain vulnerabilities in AI agent skill/plugin ecosystems. 404 Media. 2026 [accessed on 31 January 2026]. Available from: <https://www.wiz.io/blog/exposed-moltbook-database-reveals-millions-of-api-keys>
201. Willison S. The lethal trifecta for AI agents: private data, untrusted content, and external communication. Coined the term “lethal trifecta” for agents combining private data access, untrusted content exposure, and external communication. Blog post. 2025 [accessed on 15 February 2026]. Available from: <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>
202. Cohen S, Bitton R, Nassi B. Here comes the AI worm: unleashing zero-click worms that target GenAI-powered applications. 2024 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2403.02817>
203. Liu Y, Wang W, Feng R, Zhang Y, Xu G, Deng G, et al. Agent skills in the wild: an empirical study of security vulnerabilities at scale. 2026 [accessed on 15 January 2026]. Available from: <https://arxiv.org/abs/2601.10338>